

Prácticas sanas de programación

- 8.1 Al almacenar una cadena de caracteres en un arreglo de caracteres, asegúrese que el arreglo sea lo suficiente extenso para contener la cadena más grande que pueda ser almacenada. C permite que se almacenen cadenas de cualquier longitud. Si una cadena resulta más larga que el arreglo de caracteres en la cual debe almacenarse, los caracteres que excedan del final del arreglo sobrescribirán datos en memoria a continuación del mismo.
- 8.2 Al usar funciones de la biblioteca de manejo de caracteres, incluya el archivo de cabecera `<ctype.h>`.
- 8.3 Al utilizar funciones de la biblioteca general de utilerías, incluya el archivo de cabecera `<stdlib.h>`.
- 8.4 Al utilizar funciones de la biblioteca estándar de entrada/salidas, incluya el archivo de cabecera `<stdio.h>`.
- 8.5 Al utilizar funciones de la biblioteca de manejo de cadenas, incluya el archivo de cabecera `<string.h>`.

Sugerencias de portabilidad

- 8.1 Cuando una variable del tipo `char *` es inicializada con una literal de cadena, algunos compiladores pudieran colocar la cadena en una posición en memoria donde ésta no pueda ser modificada. Si pudiera necesitar modificar una literal de cadena, deberá ser almacenada en un arreglo de caracteres, para asegurarse la modificabilidad en todos los sistemas.
- 8.2 Los códigos numéricos internos utilizados para representar caracteres pudieran ser distintos en diferentes computadoras.
- 8.3 El tipo `size_t` es un sinónimo, dependiente del sistema, para el tipo `unsigned long` o para el tipo `unsigned int`.
- 8.4 El mensaje generado por `strerror` es dependiente del sistema.

Ejercicios de autoevaluación

8.1 Escriba un enunciado para ejecutar cada uno de los siguientes. Suponga que las variables `c` (que almacenan un carácter), `x`, `y` y `z` son del tipo `int`, y las variables `d`, `e`, y `f` son del tipo `float`, que la variable `ptr` es del tipo `char *`, y los arreglos `s1[100]` y `s2[100]` son del tipo `char`.

- a) Convierta en una letra mayúscula, el carácter almacenado en la variable `c`. Asigne el resultado a la variable `c`.
- b) Determine si el valor de la variable `c` es un dígito. Cuando se despliegue el resultado utilice el operador condicional, para imprimir `"is a "` o `"is not a "`, tal y como se mostró en las figuras 8.2, 8.3 y 8.4.
- c) Convierta la cadena `"1234567"` a `long` e imprima el valor.
- d) Determine si el valor de la variable `c` es un carácter de control. Cuando se despliegue el resultado utilice el operador condicional para imprimir `"is a "`, o bien `"is not a "`.
- e) Lea desde el teclado una línea de texto al arreglo `s1`. No utilice `scanf`.
- f) Imprima la línea de texto almacenado en el arreglo `s1`. No utilice `printf`.
- g) Asigne `ptr` la posición de la última instancia de `c` en `s1`.
- h) Imprima el valor de la variable `c`. No utilice `printf`.
- i) Convierta la cadena `"8.63582"` a `double` e imprima el valor.
- j) Determine si el valor de `c` es una letra. Cuando se muestre el resultado utilice el operador condicional para imprimir `"is a "`, o bien `"is not a "`.
- k) Lea un carácter del teclado, y almacene dicho carácter en la variable `c`.
- l) Asigne `ptr` la posición de la primera instancia de `s2` en `s1`.
- m) Determine si el valor de la variable `c` es un carácter de impresión. Cuando se muestre el resultado utilice el operador condicional para imprimir `"is a "`, o bien `"is not a "`.
- n) Lea tres valores `float` en las variables `d`, `e` o `f` a partir de la cadena `"1.27 10.3 9.432"`.

- o) Copie la cadena almacenada en el arreglo `s2` al arreglo `s1`.
- p) Asigne `ptr` a la posición en la primera ocurrencia de `s1`, de cualquier carácter de `s2`.
- q) Compare la cadena en `s1` con la cadena en `s2`. Imprima el resultado.
- r) Asigne `ptr` a la posición en la primera ocurrencia de `c` en `s1`.
- s) Utilice `sprintf` para imprimir los valores de las variables enteras `s`, `y` y `z` en el arreglo `s1`. Cada valor deberá ser impreso con un ancho de campo de 7.
- t) Agregue 10 caracteres de la cadena en `s2` a la cadena en `s1`.
- u) Determine la longitud de la cadena en `s1`. Imprima el resultado.
- v) Convierta la cadena `"-21"` a `int` e imprima el valor.
- w) Asigne `ptr` a la posición del primer token en `s1`. Los tokens en `s2` están separados por comas (,).

8.2 Muestre dos métodos diferentes para inicializar el arreglo de caracteres `vowel` con la cadena de vocales, `"AEIOU"`

8.3 ¿Qué es lo que, si es que existe, se imprime cuando se ejecutan cada uno de los siguientes enunciados de C? Si el enunciado contiene un error, descríbalos e indique cómo corregirlo. Suponga las siguientes declaraciones de variables:

```
char s1[50] = "jack", s2[50] = "jill", s3[50], *sptr;
```

- a) `printf("%c%s", toupper(s1[0]), &s1[1]);`
- b) `printf("%s", strcpy(s3, s2));`
- c) `printf("%s", strcat(strcat(strcpy(s3, s1), "and"), s2));`
- d) `printf("%u", strlen(s1) + strlen(s2));`
- e) `printf("%u", strlen(s3));`

8.4 Encuentre el error en cada uno de los segmentos de programas siguientes, y explique cómo corregirlo:

- a)

```
char s[10];
strncpy(s, "hello", 5);
printf("%s\n", s);
```
- b) `printf("%s", 'a');`
- c)

```
char s[12];
strcpy(s, "Welcome Home");
```
- d)

```
if (strcmp(string1, string2))
printf("The strings are equal\n");
```

Respuestas a los ejercicios de autoevaluación

- 8.1
 - a) `c = toupper(c);`
 - b) `printf("'%c'%sdigit\n", c, isdigit(c) ? " is a " : " is not a ");`
 - c) `printf("%ld\n", atol("1234567"));`
 - d) `printf("'%c'%scontrol character\n", c, iscntrl(c) ? " is a " : " is not a ");`
 - e) `gets(s1);`
 - f) `puts(s1);`
 - g) `ptr = strrchr(s1, c);`
 - h) `putchar(c);`
 - i) `printf("%f\n", atof("8.63582"));`
 - j) `printf("'%c'%sletter\n", c, isalpha(c) ? " is a " : " is not a ");`
 - k) `c = getchar();`

```

l) ptr = strstr(s1, s2);
m) printf("' %c' %sprinting character\n",
    c, isprint(c) ? " is a " : " is not a ");
n) sscanf("1.27 10.3 0.432", "%f%f%f", &d, &e, &f);
o) strcpy(s1, s2);
p) ptr = strpbrk(s1, s2);
q) printf("strcmp(s1, s2) = %d\n", strcmp(s1, s2));
r) ptr = strchr(s1, c);
s) sprintf(s1, "%7d%7d%7d", x, y, z);
t) strncat(s1, s2, 10);
u) printf("strlen(s1) = %u\n, strlen(s1));
v) printf("%d\n", atoi("-21"));
w) ptr = strtok(s2, ",");

```

8.2 char vowel[] = "AEIOU";

CHAR vowel[] = {'A', 'E', 'I', 'O', 'U', '\0'};

- 8.3 a) Jack
b) jill
c) jack and jill
d) 8
e) 13

- 8.4 a) Error: la función `strncpy` no escribe un carácter de terminación `NULL` al arreglo `s`, porque su tercer argumento es igual a la longitud de la cadena "hello".
Corrección: haga 6 el tercer argumento de `strncpy`, o asigne `'\0'` a `s[5]`.
b) Error: intentar imprimir una constante de caracteres como una cadena.
Corrección: utilice `%c` para extraer el carácter, o bien remplace `'a'` con `"a"`.
c) Error: el arreglo de caracteres `s` no tiene suficiente tamaño para almacenar el carácter de terminación `NULL`.
Corrección: Declare el arreglo con más elementos.
d) Error: la función `strcmp` regresará 0 si las cadenas son iguales, por lo tanto, será falsa la condición en la estructura `if`, y `printf` no será ejecutado.
Corrección: compare en la condición el resultado de `strcmp` con 0.

Ejercicios

8.5 Escriba un programa que introduzca un carácter del teclado, y pruebe el carácter con cada una de las funciones en la biblioteca de manejo de caracteres. El programa deberá imprimir el valor regresado por cada una de las funciones.

8.6 Escriba un programa que introduzca una línea de texto con la función `gets`, en el arreglo de caracteres `s[100]`. Extraiga la línea en letras mayúsculas y minúsculas.

8.7 Escriba un programa que introduzca 4 cadenas que representen enteros, convierta las cadenas a enteros, sume los valores e imprima el total de los 4 valores.

8.8 Escriba un programa que introduzca 4 cadenas, que representen valores en punto flotante, convierta las cadenas a valores `double`, sume los valores e imprima el total de los 4 valores.

8.9 Escriba un programa que utilice la función `strcmp`, para comparar dos cadenas introducidas por el usuario. El programa deberá indicar si la primera cadena es menor que, igual que o mayor que la segunda cadena.

8.10 Escriba un programa que utilice la función `strncmp`, para comparar dos cadenas introducidas por el usuario. El programa deberá introducir el número de caracteres a compararse. El programa deberá indicar si la primera cadena es menor que, igual que o mayor que la segunda cadena.

8.11 Escriba un programa que utilice generación de números aleatorios para crear oraciones. El programa deberá utilizar cuatro arreglos de apuntadores a `char` llamados `article`, `noun`, `verb`, y `preposition`. El programa deberá crear una oración, seleccionando una palabra al azar de cada uno de los arreglos, en el orden siguiente: `article`, `noun`, `verb`, `preposition`, `article` y `noun`. Conforme se seleccione cada palabra, deberá ser concatenada con las palabras anteriores en un arreglo lo suficiente extensa para contener a toda la oración. Las palabras deberán estar separadas por espacios. Cuando se extraiga la oración final, deberá iniciar con una letra mayúscula y terminar con un punto. El programa deberá generar 20 oraciones de este tipo.

Los arreglos deberán ser llenados como sigue: el arreglo `article` deberá contener los artículos "the", "a", "one", "some", y "any"; el arreglo `noun` deberá contener los nombres "boy", "girl", "dog", "town", y "car"; el arreglo `verb` deberá contener los verbos "drove", "jumped", "ran", "walked", y "skipped"; el arreglo `preposition` deberá contener las preposiciones: "to", "from", "over", "under", y "on".

Una vez escrito el programa anterior y funcionando, modifíquelo para producir una corta historia que esté formada con varias de estas oraciones. (¿Qué tal le parecería la posibilidad de convertirse en un escritor de tesis al azar!)

8.12 (*Quintillas jocosas*). Una quintilla jocosa es un verso cómico de cinco líneas, en las cuales la primera y segunda línea riman con la quinta, y la tercera rima con la cuarta. Utilizando técnicas similares a las desarrolladas en el ejercicio 8.10, escriba un programa en C que produzca quintillas al azar. Pulir este programa para que pueda producir buenas quintillas es un verdadero reto, ¡pero el resultado puede merecer el esfuerzo!.

8.13 Escriba un programa que cifre frases de la lengua inglesa y las convierta en latín infantil. El latín infantil es una forma de lenguaje codificado utilizado a menudo para diversión. Existen muchas variantes en los métodos utilizados para formar frases en latín infantil. Para simplificar, utilice el algoritmo siguiente:

Para formar una frase en latín infantil a partir de una frase en lengua inglesa, divida la frase en palabras, utilizando la función `strtok`. Para traducir cada palabra inglesa en una palabra en latín infantil, coloque la primera letra de la palabra inglesa al final de la palabra y añada las letras "ay". De ahí la palabra "jump" se convierte en "umpjay", la palabra "the" se convierte en "hetay", y la palabra `computer` se convierte en "omputercaay". Los espacios en blanco entre palabras se conservan como están. Suponga lo siguiente: la frase en inglés está formada de palabras separadas por espacios en blanco, no hay signos de puntuación y todas las palabras tienen dos o más letras. La función `printLatinWord` deberá desplegar cada palabra. *Sugerencia:* cada vez que se encuentre un token en una llamada a `strtok`, pase el apuntador del token a la función `printLatinWord`, e imprima la palabra en latín infantil.

8.14 Escriba un programa que introduzca un número telefónico como cadena, en la forma (555) 555-5555. El programa deberá utilizar la función `strtok` para extraer el código de área como token, los tres primeros dígitos del número telefónico como token y los últimos cuatro dígitos del número telefónico como token. Los siete dígitos del número telefónico deberán ser concatenados en una cadena. El programa deberá convertir la cadena del código de área a `int`, y convertir la cadena del número telefónico a `long`. Tanto el código de área como el número telefónico deberán ser impresos.

8.15 Escriba un programa que introduzca una línea de texto, divida la línea con la función `strtok` y extraiga los tokens en orden inverso.

8.16 Escriba un programa que introduzca desde el teclado una línea de texto y una cadena de búsqueda. Utilizando la función `strstr` localice en la línea de texto la primera ocurrencia de la cadena de búsqueda, y asigne la posición a la variable `searchPtr` del tipo `char *`. Si encuentra la cadena de búsqueda, imprima el resto de la línea de texto, empezando con la cadena de texto. A continuación utilice otra vez `strstr`, para localizar en la línea de texto la siguiente ocurrencia de la cadena de búsqueda. Si encuentra una segunda ocurrencia, imprima el resto de la línea de texto, empezando con la segunda ocurrencia. *Sugerencia:* la segunda llamada a `strstr` deberá contener `searchPtr + 1` como primer argumento.

8.17 Escriba un programa basado en el programa del ejercicio 8.16, que introduzca varias líneas de texto y una cadena de búsqueda, y utilice la función `strstr` para determinar todas las ocurrencias de la cadena, en las líneas de texto. Imprima el resultado.

8.18 Escriba un programa que introduzca varias líneas de texto y un carácter de búsqueda, y utilice la función `strchr` para determinar todas las ocurrencias del carácter, en las líneas de texto

8.19 Escriba un programa basado en el programa del ejercicio 8.18 que introduzca varias líneas de texto y utilice la función `strchr` para determinar todas las ocurrencias de cada letra del alfabeto en las líneas de texto. Las letras mayúsculas y minúsculas deberán ser contadas juntas. Almacene en un arreglo los totales de cada letra, y una vez que se hayan determinado los totales, imprímalos en formato tabular.

8.20 Escriba un programa que introduzca varias líneas de texto y utilice `strtok`, para contar el número total de palabras. Suponga que las palabras están separadas, ya sea por espacios o por caracteres de nueva línea.

8.21 Utilice las funciones de comparación de cadenas, analizadas en la Sección 8.6, y las técnicas para ordenamiento de arreglos, desarrolladas en el capítulo 6, para escribir un programa que alfabetice una lista de cadenas. Utilice como datos para su programa los nombres de 10 ó 15 ciudades en su área.

8.22 La gráfica en el apéndice D muestra las representaciones de código numérico, para los caracteres del conjunto de caracteres ASCII. Estudie esta gráfica y a continuación, indique si cada uno de los siguientes es verdadero o falso.

- a) La letra "A" viene antes de la letra "B".
- b) El dígito "9" viene antes del dígito "0".
- c) Los símbolos comúnmente utilizados para la suma, resta, multiplicación y división, todos ellos vienen antes de cualquiera de los dígitos.
- d) Los dígitos vienen antes de las letras.
- e) Si un programa de ordenación ordena cadenas en una secuencia ascendente, entonces el programa colocará el símbolo para un paréntesis derecho, antes del símbolo para uno izquierdo.

8.23 Escriba un programa que lea una serie de cadenas, e imprima sólo estas cadenas, empezando con la letra "b".

8.24 Escriba un programa que lea una serie de cadenas, e imprima sólo aquellas cadenas que terminan con las letras "ED".

8.25 Escriba un programa que introduzca un código ASCII e imprima el carácter correspondiente. Modifique este programa de tal forma que genere todos los códigos posibles de tres dígitos en el rango de 000 a 255 e intente imprimir los caracteres correspondientes. ¿Qué es lo que ocurre cuando se ejecuta este programa?

8.26 Utilizando la gráfica de caracteres ASCII del apéndice D como guía, escriba sus propias versiones de las funciones de manejo de caracteres de la figura 8.1.

8.27 Escriba sus propias versiones de las funciones en la figura 8.5 para conversión de cadenas a números.

8.28 Escriba dos versiones de cada una de las funciones de copia y concatenación de cadenas de la figura 8.17. La primera versión deberá utilizar subíndices de arreglos, y la segunda versión deberá utilizar apuntadores y aritmética de apuntadores.

8.29 Escriba sus propias versiones de las funciones `getchar`, `gets`, `putchar`, y `puts`, descritas en la figura 8.12.

8.30 Escriba dos versiones de cada una de las funciones de comparación de cadenas de la figura. 8.20. La primera versión deberá usar subíndices de arreglos, y la segunda apuntadores y aritmética de apuntadores.

8.31 Escriba sus propias versiones de las funciones en la figura 8.22, para la búsqueda de cadenas.

8.32 Escriba su propias versiones de las funciones en la figura 8.30, para manipular bloques de memoria.

8.33 Escriba dos versiones de la función `strlen` de la figura 8.36. La primera versión deberá utilizar subíndices de arreglos, y la segunda deberá utilizar apuntadores y aritmética de apuntadores.

Sección especial: ejercicios avanzados de manipulación de cadenas

Los ejercicios anteriores están relacionados con el texto y diseñados para probar la comprensión del lector sobre conceptos fundamentales de manipulación de cadenas. En esta sección se incluye un conjunto de problemas intermedios y avanzados. El lector encontrará estos problemas excitantes y divertidos. Los problemas varían de forma considerable al grado de dificultad. Algunos requieren de una hora o dos de escritura y puesta en marcha de los programas. Otros son útiles para tareas de laboratorio, que pudieran requerir dos a tres semanas para su estudio y puesta en marcha. Algunos son proyectos de tesis que plantean retos interesantes.

8.34 (Análisis de texto). La disponibilidad de computadoras con capacidades de manipulación de cadenas ha resultado en varios métodos interesantes para analizar lo escrito por grandes autores. Se ha puesto gran atención al hecho de saber si William Shakespeare alguna vez existió. Algunos estudiosos creen que existe evidencia sustancial indicando que Christopher Marlowe, de hecho fue el que escribió las obras maestras atribuidas a Shakespeare. Los investigadores han utilizado computadoras para localizar similitudes en los textos de estos dos autores. Este ejercicio examina tres métodos para analizar texto, utilizando una computadora.

- a) Escriba un programa que lea varias líneas de texto e imprima una tabla indicando el número de instancias de cada letra del alfabeto en dicho texto. Por ejemplo, la frase

To be, or not to be: that is the question:

contiene una "a", dos "b", ninguna "c", etcétera.

- b) Escriba un programa que lea varias líneas de texto e imprima una tabla que indique el número de palabras de una letra, de dos letras, de tres letras que aparecen en el texto. Por ejemplo, la frase

Whether 'tis nobler in the mind to suffer

contiene

Longitud de palabra	Ocurrencias
1	0
2	2
3	2
4	2 (including 'tis)
5	0
6	2
7	1

- c) Escriba un programa que lea varias líneas de texto e imprima una tabla indicando el número de ocurrencias de cada palabra distinta en el texto. La primera versión de su programa deberá incluir en la tabla las palabras en el mismo orden en que aparecen dentro de texto. Una impresión más interesante (y más útil) deberá intentarse, en la cual las palabras se ordenarán en forma alfabética. Por ejemplo, las líneas

To be, or not to be: that is the question:
Whether 'tis nobler in the mind to suffer

contiene las palabras "to" tres veces, la palabra "be" dos veces, la palabra "or" una vez, etcétera.

8.35 (*Procesamiento de palabras*). El tratamiento detallado de la manipulación de cadenas en este texto debe atribuirse principalmente al excitante crecimiento, en años recientes, del procesamiento de texto. Una función importante de los sistemas de procesamiento de palabras es el *tipo justificado* —la alineación de las palabras, tanto en los márgenes izquierdo como derecho de una página. Esto genera un documento de apariencia profesional, que parece haber sido formado en tipografía, en vez de preparado en una máquina de escribir. El tipo justificado puede ser llevado a cabo en sistemas de computación, insertando uno o más caracteres en blanco entre cada una de las palabras de una línea, de tal forma que la palabra más a la derecha se alinee con el margen derecho.

Escriba un programa que lea varias líneas de texto e imprima este texto en formato de tipo justificado. Suponga que el texto debe ser impreso en papel de un ancho de 8 1/2 pulgadas, y que se deben dejar márgenes de una pulgada, tanto en el lado derecho como izquierdo de la página impresa. Suponga que la computadora imprime 10 caracteres por cada pulgada horizontal. Por lo tanto, su programa deberá imprimir 6 1/2 pulgadas de texto, es decir, 65 caracteres por línea.

8.36 (*Impresión de fechas en varios formatos*). En la correspondencia de negocios, las fechas se imprimen comúnmente en varios formatos diferentes. Dos de los formatos más comunes son:

07/21/55 and July 21, 1955

Escriba un programa que lea una fecha en el primer formato y la imprima en el segundo.

8.37 (*Protección de cheques*). Con frecuencia las computadoras se utilizan en sistemas de escritura de cheques, como son aplicaciones de nóminas y cuentas por pagar. Circulan muchas historias raras, en relación con la impresión de nóminas semanales impresas (por error) por cantidades en exceso de 1 millón. Debido a fallas humanas o de máquina, cantidades rarísimas han sido impresas por sistemas computarizados de escritura de cheques. Los diseñadores de estos sistemas, naturalmente, hacen todo tipo de esfuerzo para incorporar controles en sus sistemas, a fin de evitar que se emitan cheques equivocados.

Otro problema serio es la alteración intencional de la cantidad en el cheque, por alguien que intenta cobrar un cheque de forma fraudulenta. Para evitar que sea alterada una cantidad en dólares, la mayor parte de los sistemas computarizados de escritura de cheques emplean una técnica conocida como *protección de cheques*.

Los cheques diseñados para impresión por computadora contienen un número fijo de espacios, en el cual la computadora puede imprimir una cantidad. Suponga que un cheque de nómina contiene ocho espacios en blanco, en el cual la computadora se supone imprimirá la cantidad de la nómina semanal. Si la cantidad es grande, entonces todos los ocho espacios quedarán llenos, por ejemplo:

1,230.60 (check amount)
12345678 (position numbers)

Por otra parte, si la cantidad es menor de \$1000, entonces varios de los espacios quedarían en blanco. Por ejemplo,

99.87
12345678

contiene tres espacios en blanco. Si un cheque se imprime con espacios en blanco, es más fácil que alguien pueda alterar la cantidad del cheque. A fin de evitar que se modifique un cheque, muchos sistemas de escritura de cheques *presentan asteriscos*, para proteger la cantidad, como sigue:

***99.87
12345678

Escriba un programa que introduzca una cantidad en dólares a imprimirse en un cheque, y a continuación imprima en formato protegido de cheque con asteriscos anteriores, si ello fuera necesario. Suponga que para la impresión de una cantidad están disponibles nueve espacios.

8.38 (*Escribir el equivalente en palabras de una cantidad de cheques*). Continuando con el análisis del ejemplo anterior, insistimos en la importancia de diseñar sistemas de escritura de cheques, que impidan la modificación de las cantidades de los cheques. Un método común de seguridad requiere que la cantidad de cheque se escriba tanto en números como en palabras. Aun si alguien es capaz de modificar la cantidad numérica del cheque, resulta en extremo, difícil modificar la cantidad en palabras.

Muchos sistemas computarizados de escritura de cheques no incluyen la cantidad del cheque en palabras. Quizás la razón principal de esta omisión es el hecho de que la mayoría de los lenguajes de alto nivel utilizados en aplicaciones comerciales, no contienen características adecuadas de manipulación de cadenas. Otra razón es que la lógica para la escritura de equivalentes en palabras de las cantidades de los cheques es algo compleja.

Escriba un programa en C que introduzca una cantidad de cheque numérico y escriba el equivalente en palabras de dicha cantidad. Por ejemplo, la cantidad 112.43 deberá quedar escrita como

ONE HUNDRED TWELVE and 43/100

8.39 (*Código Morse*). Quizás el más famoso de todos los sistemas de codificación es el código Morse, desarrollado por Samuel Morse en 1832, para uso en el sistema telegráfico. El código Morse asigna una serie de puntos y rayas a cada letra del alfabeto, a cada dígito y a unos cuantos caracteres especiales (como el punto, coma, punto y coma; y dos puntos). En los sistemas orientados a sonido, el punto representa un sonido corto y la raya representa un sonido largo. Se utilizan otras representaciones de los puntos y rayas, con sistemas orientados a luz y sistemas de señalización por banderas.

La separación entre palabras se indica por un espacio, o por la ausencia de un punto o de una raya. En un sistema orientado a sonidos, un espacio queda indicado por un corto periodo de tiempo, en el cual ningún sonido se transmite. La versión internacional del código Morse aparece en la figura 8.39.

Escriba un programa que lea una frase en lengua inglesa y que cifre la frase en código Morse. También escriba un programa que lea una frase en código Morse y la convierta en el equivalente en lengua inglesa. Utilice un espacio en blanco entre cada letra codificada Morse y tres espacios en blanco entre cada palabra codificada en Morse.

8.40 (*Programa de conversión métrica*). Escriba un programa que ayude al usuario con las conversiones métricas. Su programa deberá permitir al usuario especificar como cadenas los nombres de las unidades (es decir, centímetros, litros, gramos, etcétera para el sistema métrico y pulgadas, cuartos, libras, etcétera para el sistema inglés) y deberá responder a preguntas sencillas como

"¿How many inches are in 2 meters?"
"¿How many liters are in 10 quarts?"

Su programa deberá poder reconocer conversiones inválidas. Por ejemplo, la pregunta

"¿How many feet in 5 kilograms?"

no tiene sentido, porque "feet" son unidades de longitud, en tanto que "kilogram" es una unidad de peso.

8.41 (*Cartas de cobranza*). Muchos negocios gastan mucho tiempo y dinero cobrando deudas vencidas. La cobranza de cuentas vencidas es el proceso de efectuar demandas repetidas o insistentes por carta a un deudor, en un intento de cobrar una deuda.

A menudo se utilizan computadoras para generar automáticamente cartas de cobranza, en grado creciente de severidad, conforme la cuenta se hace más vieja. La teoría es que mientras más vieja sea la cuenta, más difícil será su cobranza, por lo tanto, las cartas correspondientes deberán ser más y más amenazadoras.

Carácter	Código	Carácter	Código
A	.-	T	-
B		U	..-
C	...-	V	...-
D	...-	W	...-
E	.	X	...-
F	...-	Y	...-
G	...-	Z	...-
H			
I	..	Digits	
J	...-	1	-
K	...-	2	-
L	...-	3	-
M	...-	4	-
N	...-	5	-
O	...-	6	-
P	...-	7	-
Q	...-	8	-
R	...-	9	-
S	...-	0	-

Fig. 8.39 Las letras del alfabeto tal y como se expresan en el código internacional Morse.

Escriba un programa en C que contenga los textos de cinco cartas de cobranza, de severidad creciente. Su programa deberá aceptar como entrada:

- 1. El nombre del deudor.
- 2. La dirección del deudor.
- 3. La cuenta del deudor.
- 4. La cantidad adeudada.
- 5. El atraso en la cantidad adeudada (es decir, un mes de vencida, dos meses de vencida, etcétera).

Utilice el atraso en el pago para seleccionar uno de los cinco textos de mensaje, y a continuación imprima la carta de cobranza, insertando donde resulte apropiada la otra información proporcionada por el usuario.

Un proyecto de manipulación de cadenas que resulta un reto

8.42 (Un generador de palabras cruzadas). La mayor parte de las personas alguna vez han resuelto un crucigrama, pero pocos han intentado generar uno. Generar un crucigrama es un problema difícil. Se sugiere aquí como un proyecto de manipulación de cadenas, que requiere complejidad y esfuerzo sustancial. Existen muchos ángulos que el programador deberá resolver, para conseguir que funcione, inclusive el más simple

de los programas de generación de crucigramas. Por ejemplo, ¿cómo representa la cuadrícula de un crucigrama en la computadora? ¿Deberá utilizarse una serie de cadenas, o arreglos de doble subíndice? El programador necesitará una fuente de palabras (es decir, un diccionario computarizado) que pueda ser consultado de forma directa por el programa. ¿En qué forma deberán ser almacenadas estas palabras, para facilitar las manipulaciones complejas requeridas por el programa? El lector de verdad ambicioso deseará generar la porción de “indicios o pistas” en la cual se imprimen las breves sugerencias para cada palabra “horizontal” y cada palabra “vertical” para quien está resolviendo el crucigrama. Simplemente imprimir una versión del crucigrama en blanco por sí mismo no resulta un problema sencillo.

9

Entradas/salidas con formato

Objetivos

- Comprender flujos de entrada y de salida.
- Ser capaz de utilizar todas las capacidades de formato de impresión.
- Ser capaz de utilizar todas las capacidades de formato de entrada.

Todas las noticias posibles de imprimir.

Adolph S. Ochs

¿Qué loca quimera? ¿Qué lucha por escapar?

John Keats

No quiten las mojoneras en los límites de los campos.

Amenemope

El fin debe de justificar los medios.

Matthew Prior

Sinopsis

- 9.1 Introducción
- 9.2 Flujos
- 9.3 Salida con formato utilizando printf
- 9.4 Cómo imprimir enteros
- 9.5 Cómo imprimir números de punto flotante
- 9.6 Cómo imprimir cadenas y caracteres
- 9.7 Otros especificadores de conversión
- 9.8 Cómo imprimir con anchos de campo y precisiones
- 9.9 Uso de banderas en la cadena de control de formato de printf
- 9.10 Cómo imprimir literales y secuencias de escape
- 9.11 Formato de entrada utilizando scanf

Resumen • Terminología • Errores comunes de programación • Prácticas sanas de programación • Sugerencias de portabilidad • Ejercicios de autoevaluación • Respuestas a los ejercicios de autoevaluación • Ejercicios.

9.1 Introducción

Una parte importante de la solución de cualquier problema es la presentación de los resultados. En este capítulo analizamos a fondo las características de formato de `printf` y de `scanf`. Estas funciones introducen flujos de datos estándar de entrada y extraen flujos de datos estándar de salida, respectivamente. Las otras cuatro funciones que utilizan entradas y salidas estándar `gets`, `puts`, `getchar`, y `putchar` fueron analizadas en el capítulo 8. Incluya el archivo de cabecera `<stdio.h>` en aquellos programas que llamen a estas funciones.

Muchas características de `printf` y de `scanf` han sido ya analizadas en el texto. Este capítulo resume estas características y presenta muchas otras. En el capítulo 11 se analizarán varias otras funciones incluidas en la biblioteca estándar de entrada/salidas (`stdio`).

9.2 Flujos

Toda entrada y salida se ejecuta mediante flujos —secuencias de caracteres organizadas en líneas. Cada línea está formada de cero o más caracteres y está terminada por el carácter de nueva línea. El estándar indica que las aplicaciones de ANSI C deben soportar líneas de por lo menos 254 caracteres, incluyendo un carácter de nueva línea de terminación.

Cuando empieza la ejecución del programa, de forma automática se conectan tres flujos al programa. Por lo regular, el flujo estándar de entrada se conecta al teclado y el flujo estándar de salida se conecta a la pantalla. Con frecuencia los sistemas operativos permiten que estos flujos sean redirigidos a otros dispositivos. Un tercer flujo, el error estándar, se conecta a la pantalla.

Los mensajes de error son extraídos o desplegados al flujo estándar de error. Los flujos se analizan en detalle en el capítulo 11, “Procesamiento de archivos”.

9.3 Salida con formato utilizando printf

Utilizando `printf` se consigue un formato de salida preciso. Cada llamada `printf` contiene una cadena de control de formato, que describe el formato de la salida. La cadena de control de formato consiste de especificadores de conversión, banderas, anchos de campo, precisiones y caracteres literales. Junto con el signo de por ciento (%), éstos forman especificaciones de conversión. La función `printf` puede llevar a cabo las siguientes capacidades de formato, cada una de las cuales se analiza en este capítulo.

- 1 Redondear valores de punto flotante, a un número indicado de valores decimales.
- 2 Alinear una columna de números, con puntos decimales apareciendo uno por encima del otro.
- 3 Salidas justificadas a la derecha o a la izquierda.
- 4 Insertar caracteres literales en posiciones precisas en una línea de salida.
- 5 Representación en formato exponencial de números de punto flotante.
- 6 Representación en formato octal y hexadecimal de enteros no signados. Vea el apéndice E, “Sistemas numéricos” para más información sobre valores octales y hexadecimales.
- 7 Despliegue de todo tipo de datos con anchos de campo de tamaño fijo y precisiones.

La función `printf` tiene la forma:

```
printf (format-control-string, other-arguments);
```

Format-control-string describe el formato de salida, y *other-arguments* (estos son opcionales) corresponden a cada especificación de conversión existente en *format-control-string*. Cada especificación de conversión se inicia con un signo de por ciento y termina con un especificador de conversión. En una cadena de control de formato pueden existir muchas especificaciones de conversión.

Error común de programación 9.1

Olvidar encerrar entre comillas una cadena de control de formato.

Práctica sana de programación 9.1

Edite las salidas de manera nítida para su presentación. Esto hace más legible las salidas del programa y reduce errores de usuario.

9.4 Cómo imprimir enteros

Un entero es un número completo, como 776 o -52, que no contiene punto decimal. Los valores enteros se despliegan en uno de varios formatos. En la figura 9.1 se describe cada uno de los especificadores de conversión de enteros.

El programa de la figura 9.2 imprime un entero, utilizando cada uno de los especificadores de conversión de enteros. Note que sólo se imprime el signo menos; los signos más son suprimidos. Más adelante en este capítulo, veremos cómo obligar a que los signos más se impriman. También note que en una computadora con enteros de 2 bytes, el valor -455 es leído por `%u` y se convierte al valor no signado 65081.

Especificador de conversión	Descripción
d	Despliega un entero decimal signado.
i	Muestra un entero decimal signado. (Nota: los especificadores i y d son diferentes cuando se utilizan con scanf).
o	Muestra un entero octal no signado.
u	Muestra un entero decimal no signado.
x o bien X	Muestra un entero hexadecimal no signado. X mayúscula hace que sean mostrados los dígitos 0-9 y las letras A-F, y x minúsculas hacen que se muestren los dígitos 0-9 y a-f.
h o bien l (letra l)	Se coloca antes de cualquier especificador de conversión de enteros para indicar que se muestra un entero short, o bien long respectivamente.

Fig. 9.1 Especificadores de conversión de enteros.

```
/* Using the integer conversion specifiers */
#include <stdio.h>

main()
{
    printf("%d\n", 455);
    printf("%i\n", 455); /* i same as d in printf */
    printf("%d\n", +455);
    printf("%d\n", -455);
    printf("%hd\n", 32000);
    printf("%ld\n", 2000000000);
    printf("%o\n", 455);
    printf("%u\n", 455);
    printf("%u\n", -455);
    printf("%x\n", 455);
    printf("%X\n", 455);
    return 0;
}
```

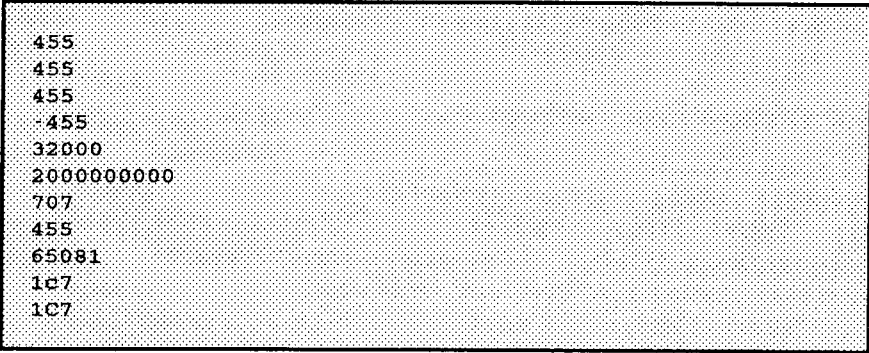


Fig. 9.2 Cómo utilizar especificadores de conversión de enteros.

Error común de programación 9.2

Imprimir un valor negativo utilizando un especificador de conversión que espera un valor no signado.

9.5 Cómo imprimir números de punto flotante

Un valor de punto flotante contiene un punto decimal, como en 33.5 o 657.983. Los valores de punto flotante son desplegados en uno de varios formatos. En la figura 9.3 se describen los especificadores de conversión de punto flotante.

Los especificadores de conversión e y E despliegan valores de punto flotante en notación exponencial. La notación exponencial es el equivalente de computadora de la notación científica utilizada en matemáticas. Por ejemplo, el valor 150.4582 se representa en notación científica como

1.504582 × 10²

y se representa en notación exponencial como

1.504582E+02

por la computadora. Esta notación indica que 1.504582 debe ser multiplicado por 10 elevado a la segunda potencia (E+02). La E significa “exponente”.

En forma preestablecida, los valores impresos con los especificadores de conversión e, E, y f son extraídos con 6 dígitos de precisión a la derecha del punto decimal; otras precisiones pueden ser especificadas de forma explícita. El especificador de conversión f siempre imprime por lo menos un dígito a la izquierda del punto decimal. Los especificadores de conversión e, y E imprimen respectivamente la e en minúsculas y la E en mayúsculas antes del exponente y siempre se imprimen exactamente un dígito a la izquierda del punto decimal.

El especificado de conversión g (G) imprime en formato e (E), o en formato f, sin ceros después del punto decimal (es decir, 1.234000 se imprime como 1.234). Los valores se imprimen con e (E), si después de convertir el valor a notación exponencial, el exponente del valor es menor de -4, o es mayor o igual a la precisión especificada (por omisión, en el caso de g y de G, 6 dígitos significativos). De lo contrario, para imprimir el valor se utilizará el especificador de conversión f. Utilizando g o G, no se imprimen los ceros a la derecha del punto decimal, en la parte fraccionaria de la salida del valor. Para que se extraiga el punto decimal se requiere por lo menos de un dígito decimal.

Especificador de conversión	Descripción
e o bien E	Muestra un valor en punto flotante en notación exponencial.
f	Muestra valores en punto flotante.
g o bien G	Despliega un valor en punto flotante, ya sea en la forma de punto flotante f, o en la forma exponencial e (o E).
L	Se coloca antes de cualquier especificador de conversión de punto flotante, para indicar que está mostrado un valor de punto flotante long double.

Fig. 9.3 Especificadores de conversión de punto flotante.

Si se utiliza la especificación de conversión **%g**, los valores **0.0000875**, **8750000.0**, **8.75**, **87.50** y **875** se imprimen como **8.75e-05**, **8.75e+06**, **8.75**, **87.5** y **875**. El valor **0.0000875** utiliza notación **e** porque, al convertirse a notación exponencial, su exponente es menor que **-4**. El valor **8750000.0** utiliza notación **e** debido a que su exponente es igual a la precisión por omisión.

La precisión de los especificadores de conversión **g** y **G** indican el número máximo de dígitos significativos impresos, incluyendo el dígito a la izquierda del punto decimal. Utilizando la especificación de conversión **%g**, el valor **1234567.0** se imprime como **1.23457e+06** (recuerde que todos los especificadores de conversión de punto flotante tienen una precisión por omisión de 6). Note que en el resultado aparecen 6 dígitos significativos. La diferencia entre **g** y **G** es idéntica a la diferencia entre **e** y **E**, cuando el valor se imprime en notación exponencial la **g** minúscula hace que la salida sea una **e**, y la **G** hace que la salida sea una **E**.

Práctica sana de programación 9.2

Al extraer datos, asegúrese que el usuario está consciente de situaciones en las cuales, debido al formato, los datos pudieran resultar imprecisos (por ejemplo, errores de redondeo debidos a precisiones especificadas).

El programa de la figura 9.4 demuestra cada una de las tres especificaciones de conversión de punto flotante. Advierta que las especificaciones de conversión **%E** y **%g** hacen que en la salida el valor se redondee.

```
/* Printing floating-point numbers with
   floating-point conversion specifiers */

#include <stdio.h>

main()
{
    printf("%e\n", 1234567.89);
    printf("%e\n", +1234567.89);
    printf("%e\n", -1234567.89);
    printf("%E\n", 1234567.89);
    printf("%f\n", 1234567.89);
    printf("%g\n", 1234567.89);
    printf("%G\n", 1234567.89);

    return 0;
}
```

```
1.234568e+06
1.234568e+06
-1.234568e+06
1.234568E+06
1234567.890000
1.23457e+06
1.23457E+06
```

Fig. 9.4 Cómo utilizar especificadores de conversión de punto flotante.

9.6 Cómo imprimir cadenas y caracteres

Los especificadores de conversión **c** y **s** se utilizan para imprimir respectivamente caracteres individuales y cadenas. El especificador de conversión **c** requiere de un argumento **char**. El especificador de conversión **s** requiere de un apuntador a **char** como argumento. El especificador de conversión **s** hace que se impriman caracteres hasta que encuentre un carácter de terminación **NULL** (**'\0'**). El programa que se muestra en la figura 9.5 despliega caracteres y cadenas con los especificadores de conversión **c** y **s**.

Error común de programación 9.3

*Utilizar **%c** para imprimir el primer carácter de una cadena. La especificación de conversión **%c** espera un argumento **char**. Una cadena es un apuntador a **char**, es decir, a **char ***.*

Error común de programación 9.4

*Utilizar **%s** para imprimir un argumento **char**. La especificación de conversión **%s** espera un argumento de tipo apuntador a **char**. En algunos sistemas, esto causará un error fatal en tiempo de ejecución, conocido como violación de acceso.*

Error común de programación 9.5

Es un error de sintaxis usar comillas sencillas alrededor de cadenas de caracteres. Las cadenas de caracteres deben estar encerradas entre comillas dobles.

Error común de programación 9.6

*Usar comillas dobles alrededor de una constante de carácter. Esto, de hecho, crea una cadena formado por dos caracteres, el segundo de los cuales es el carácter de terminación **NULL**. Una constante de carácter es un carácter, encerrado entre comillas sencillas.*

```
/* Printing strings and characters */
#include <stdio.h>

main()
{
    char character = 'A';
    char string[] = "This is a string";
    char *stringPtr = "This is also a string";

    printf("%c\n", character);
    printf("%s\n", "This is a string");
    printf("%s\n", string);
    printf("%s\n", stringPtr);
    return 0;
}
```

```
A
This is a string
This is a string
This is also a string
```

Fig. 9.5 Cómo utilizar los especificadores de conversión de caracteres y de cadenas.

9.7 Otros especificadores de conversión

Los tres especificadores de conversión restantes son **p**, **n** y **%** (figura 9.6).

Sugerencia de portabilidad 9.1

El especificador de conversión **p** despliega una dirección de apuntador en forma de puesta en marcha definida (en muchos sistemas se utiliza notación hexadecimal en preferencia a notación decimal).

El especificador de conversión **n** almacena el número de caracteres ya extraídos en el enunciado actual **printf** — el argumento correspondiente es un apuntador a una variable entera en la cual se almacena el valor. Mediante una especificación de conversión **%n** nada se imprime. El especificador de conversión **%** hace que se extraiga un signo de **%**.

En el programa de la figura 9.7, **%p** imprime el valor de **ptr** y la dirección de **x**; estos valores resultan idénticos porque a **ptr** es asignada a la dirección de **x**. A continuación, **%n** almacena el número de caracteres extraídos por el tercer enunciado **printf** en la variable entera **y**, y es impreso el valor de **y**. El último enunciado **printf** utiliza **%%** para imprimir el carácter **%** en una cadena de carácter. Note que cada una de las llamadas **printf** regresa un valor ya sea el número de caracteres extraídos o si ha ocurrido un error de salida, un valor negativo.

Error común de programación 9.7

Intentar imprimir un carácter literal de por ciento, utilizando en la cadena de control de formato **%** en vez de **%%**. Cuando en la cadena de control de formato aparece el signo de **%** debe estar seguido por un especificador de conversión.

9.8 Cómo imprimir con anchos de campo y precisiones

El tamaño exacto de un campo en el cual se imprimen datos se especifica por el *ancho del campo*. Si el ancho del campo es mayor que los datos que se están imprimiendo, a menudo los datos dentro de dicho campo quedarán justificados a la derecha. Un entero que representa el ancho del campo es insertado en la especificación de conversión, entre el signo de por ciento (**%**) y el especificador de conversión. El programa en la figura 9.8 imprime dos grupos cada uno de cinco números, justificando a la derecha aquellos números que contienen menos dígitos que el ancho del campo. Note que el ancho de campo es de manera automática aumentado para imprimir valores más anchos que el campo, y que el signo de menos de un valor negativo ocupa en el ancho de campo

Especificador de conversión	Descripción
p	Muestra un valor de apuntador en forma de puesta en marcha definida.
n	Almacena el número de caracteres ya extraídos en el enunciado printf actual. Se proporciona un apuntador a un entero como argumento correspondiente. No se muestra nada.
%	Muestra el carácter de por ciento.

Fig. 9.6 Otros especificadores de conversión.

```
/* Using the p, n, and % conversion specifiers */
#include <stdio.h>

main()
{
    int *ptr;
    int x = 12345, y;

    ptr = &x;
    printf("The value of ptr is %p\n", ptr);
    printf("The address of x is %p\n\n", &x);

    printf("Total characters printed on this line is:%n", &y);
    printf(" %d\n\n", y);

    y = printf("This line has 28 characters\n");
    printf("%d characters were printed\n\n", y);

    printf("Printing a %% in a format control string\n");
    return 0;
}
```

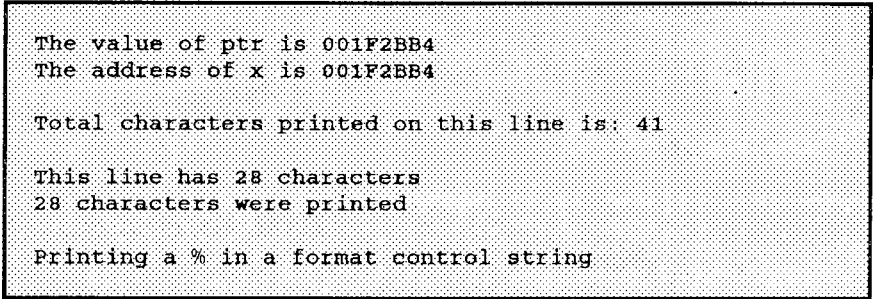


Fig. 9.7 Cómo utilizar los especificadores de conversión **p**, **n** y **%**.

una posición de carácter. Los anchos de campo pueden ser utilizados con todos los especificadores de conversión.

Error común de programación 9.8

No proporcionar un ancho de campo lo suficiente extenso para manejar un valor a imprimirse. Esto puede desplazar otros datos imprimiéndose y puede producir salidas confusas. ¡Familiarícese con sus datos!

La función **printf** también da la capacidad de definir la *precisión* con la cual los datos se imprimirán. La precisión tiene significados distintos para diferentes tipos de datos. Cuando se utiliza con especificadores de conversión de enteros, la precisión indica el número mínimo de dígitos a imprimirse. Si el valor impreso contiene menos dígitos que la precisión especificada, al valor impreso se le antepondrán ceros, hasta que el número total de dígitos sea equivalente a la precisión. La precisión por omisión para enteros es 1. Cuando se utiliza con especificadores de conversión de punto flotante **e**, **E** y **f**, la precisión es el número de dígitos que aparecerá después del punto decimal. Cuando se utilice con los especificadores de conversión **g** y **G**, la precisión es

```

/* Printing integers right-justified */
#include <stdio.h>

main()
{
    printf("%4d\n", 1);
    printf("%4d\n", 12);
    printf("%4d\n", 123);
    printf("%4d\n", 1234);
    printf("%4d\n\n", 12345);

    printf("%4d\n", -1);
    printf("%4d\n", -12);
    printf("%4d\n", -123);
    printf("%4d\n", -1234);
    printf("%4d\n", -12345);

    return 0;
}

```

```

    1
   12
  123
 1234
12345

  -1
 -12
-123
-1234
-12345

```

Fig. 9.8 Justificación de enteros a la derecha en un campo.

el número máximo de dígitos significativos a imprimirse. Cuando se utiliza con el especificador de conversión **s**, la precisión es el número máximo de caracteres a escribirse a partir de una cadena. Para utilizar la precisión, coloque un punto decimal (.) seguido por un entero que representa la precisión, entre el signo de por ciento y el especificador de conversión. El programa de la figura 9.9 demuestra el uso de la precisión en cadenas de control de formato. Note que cuando un valor en punto flotante se imprime con una precisión menor en el valor que el número original de decimales, el valor quedará redondeado.

Se pueden combinar el ancho de campo y la precisión, colocando el ancho de campo seguido por el punto decimal, seguido por la precisión, entre el signo de por ciento y el especificador de conversión, como en el enunciado

```
printf("%9.3f", 123.456789);
```

que despliega **123.4567** con 3 dígitos a la derecha del punto decimal, y queda justificado a la derecha, en un campo de 9 dígitos.

```

/* Using precision while printing integers,
   floating-point numbers, and strings */
#include <stdio.h>

main()
{
    int i = 873;
    float f = 123.94536;
    char s[] = "Happy Birthday";

    printf("Using precision for integers\n");
    printf("\t%.4d\n\t%.9d\n\n", i, i);
    printf("Using precision for floating-point numbers\n");
    printf("\t%.3f\n\t%.3e\n\t%.3g\n\n", f, f, f);
    printf("Using precision for strings\n");
    printf("\t%.11s\n", s);
    return 0;
}

```

```

Using precision for integers
      0873
    000000873

Using precision for floating-point numbers
      123.945
    1.239e+02
      124

Using precision for strings
    Happy Birth

```

Fig. 9.9 Cómo usar precisiones para mostrar información de varios tipos.

Utilizando expresiones enteras en la lista de argumentos, a continuación de la cadena de control de formato, es posible especificar el ancho de campo y la precisión. Para utilizar esta característica, inserte un ***** (asterisco) en el lugar del ancho de campo o de la precisión (o de ambos). El argumento coincidente en la lista de argumentos se evalúa y se utiliza en lugar del asterisco. El valor del argumento puede ser negativo para el ancho de campo, pero debe ser positivo para la precisión. Un valor negativo para el ancho de campo hace que la salida se justifique a la izquierda en el campo, tal y como se describe en la sección siguiente. El enunciado

```
printf("%*.*f", 7, 2, 98.736);
```

utiliza **7** para el ancho de campo, **2** para la precisión y extrae el valor **98.74** justificado a la derecha.

9.9 Uso de banderas en la cadena de control de formato de printf

La función **printf** tiene también *banderas* para complementar sus capacidades de formato de salida. Están disponibles para el usuario cinco banderas, para uso en cadenas de control de formato (figura 9.10).

Bandera	Descripción
- (signo de menos)	Justificación a la izquierda de la salida, dentro del campo especificado.
+ (signo más)	Despliega un signo más, antes de valores positivos y un signo menos, antes de valores negativos.
espacio	Imprime un espacio antes de un valor positivo que no se imprima con la bandera +.
#	Antecede un 0 al valor extraído, cuando se utiliza con el especificador de conversión octal o. Antecede 0x o 0X al valor de salida, cuando se utiliza con los especificadores de conversión hexadecimales x o X. Obliga a un punto decimal para un número de punto decimal impreso con e, E, f, g o G, que no contenga una parte fraccionaria. (Por lo regular sólo se imprime el punto decimal si le sigue algún dígito). En el caso de los especificadores g y G, no se eliminan los ceros a la derecha.
0 (cero)	Rellena un campo con ceros a la izquierda.

Fig. 9.10 Banderas de cadenas de control de formato.

Para utilizar una bandera en una cadena de control de formato, colóquela de inmediato a la derecha del signo de por ciento. En una especificación de conversión se pueden combinar varias banderas.

El programa de la figura 9.11 demuestra la justificación a la derecha y a la izquierda de una cadena, de un entero, de un carácter y de un número de punto flotante.

El programa de la figura 9.12 imprime un número positivo y uno negativo, cada uno de ellos con y sin la bandera +. Note que en ambos casos el signo de menos queda desplegado, pero el signo de más sólo es desplegado cuando se utiliza la bandera más.

El programa de la figura 9.13 antepone un espacio al número positivo con la bandera de espacio. Esto resulta útil para alinear números positivos y negativos con el mismo número de dígitos.

```
/* Right justifying and left justifying values */
#include <stdio.h>

main()
{
    printf("%10s%10d%10c%10f\n\n", "hello", 7, 'a', 1.23);
    printf("%-10s%-10d%-10c%-10f\n", "hello", 7, 'a', 1.23);
    return 0;
}
```

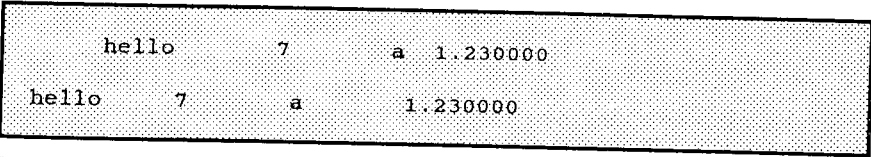


Fig. 9.11 Cómo justificar a la izquierda cadenas en un campo.

```
/* Printing numbers with and without the + flag */
#include <stdio.h>

main()
{
    printf("%d\n%d\n", 786, -786);
    printf("%+d\n%+d\n", 786, -786);
    return 0;
}
```

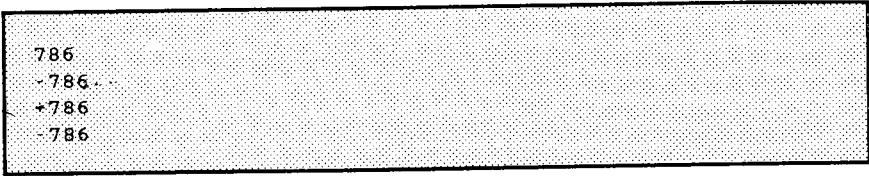


Fig. 9.12 Cómo imprimir números positivos y negativos con y sin la bandera +.

```
/* Printing a space before signed values
not preceded by + or - */
#include <stdio.h>

main()
{
    printf("% d\n% d\n", 547, -547);
    return 0;
}
```

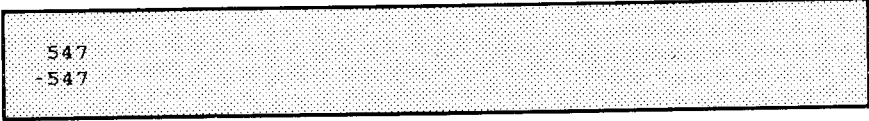


Fig. 9.13 Cómo utilizar la bandera espacio.

El programa de la figura 9.14 utiliza la bandera # para anteceder 0 al valor octal, 0x y 0X a valores hexadecimales, y para obligar a el punto decimal en un valor impreso utilizando a g.

El programa en la figura 9.15 combina la bandera + y la bandera 0 (cero) para imprimir 452 en un campo de 9 espacios, con un signo de + y ceros a la izquierda, y a continuación vuelve a imprimir 452 utilizando sólo la bandera 0 y el campo de 9 espacios.

9.10 Cómo imprimir literales y secuencias de escape

La mayor parte de los caracteres literales a imprimirse en un enunciado printf pueden ser incluidos en la cadena de control de formato. Sin embargo, existen varios caracteres "problema" tal y como los signos de comillas ("), que delimitan la cadena de control de formato misma. Varios caracteres de control, como son la nueva línea y el tabulador, deben ser representados por secuencias de escape. Una secuencia de escape es representada por una diagonal invertida (\) seguida por un carácter de escape particular. La tabla en la figura 9.16 enlista todas las secuencias de escape y las acciones que éstas causan.

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE...

```
/* Using the # flag with conversion specifiers
   o, x, X, and any floating-point specifier */
#include <stdio.h>

main()
{
    int c = 1427;
    float p = 1427.0;

    printf("%#o\n", c);
    printf("%#x\n", c);
    printf("%#X\n", c);
    printf("\n#g\n", p);
    printf("%#g\n", p);
    return 0;
}
```

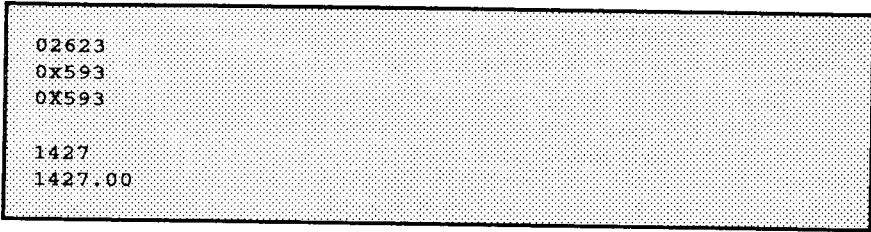


Fig. 9.14 Cómo utilizar la bandera #.

```
/* Printing with the 0(zero) flag fills in leading zeros */
#include <stdio.h>

main()
{
    printf("%+09d", 452);
    printf("%09d", 452);
    return 0;
}
```

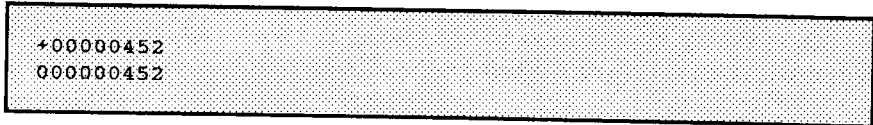


Fig. 9.15 Cómo utilizar la bandera 0 (cero).

Error común de programación 9.9

Intentar imprimir como datos literales en un enunciado printf una comilla, dobles comillas, signo de interrogación o un carácter de diagonal invertida, sin anteceder dichos caracteres por una diagonal invertida para formar una secuencia de escape correcta.

Secuencia de escape	Descripción
\'	Salida del carácter de una sola comilla (').
\"	Salida del carácter de dobles comillas (").
\?	Salida del signo de interrogación (?).
\\	Salida del carácter de diagonal invertida (\).
\a	Genera una alerta visual o audible (campana).
\b	Mueve el cursor hacia atrás una posición en la línea actual.
\f	Mueve el cursor al inicio de la siguiente página lógica.
\n	Mueve el cursor al inicio de la línea siguiente.
\r	Mueve el cursor al principio de la línea actual.
\t	Mueve el cursor a la siguiente posición de tabulador horizontal.
\v	Mueve el cursor a la siguiente posición de tabulador vertical.

Fig. 9.16 Secuencias de escape.

9.11 Formato de entrada utilizando scanf

Se consigue entrada con formato preciso utilizando scanf. Cada enunciado scanf contiene una cadena de control de formato que describe el formato de los datos que se introducen. La cadena de control está formada de especificaciones de conversión y de caracteres literales. La función scanf tiene las siguientes capacidades de formato de entrada:

- 1. Entrada de todo tipo de datos.
- 2. Entrada de caracteres específicos desde un flujo de entrada.
- 3. Omitir caracteres específicos del flujo de entrada.

La función scanf se escribe en la forma siguiente:

```
scanf (format-control-string, other-arguments) ;
```

Format-control-string describe los formatos de la entrada, y other-arguments son apuntadores a variables, en los cuales se almacena la entrada.

Práctica sana de programación 9.3

Al introducir datos, solicite al usuario un elemento o pocos elementos de datos a la vez. Evite solicitar al usuario la introducción de muchos elementos de datos, en respuesta a una solicitud.

En la figura 9.17 se resumen los especificadores de conversión utilizados para introducir todos los tipos de datos. El resto de esta sección presenta programas que demuestran la lectura de datos utilizando los varios especificadores de conversión de scanf.

El programa en la figura 9.18 lee enteros con varios especificadores de conversión de enteros, y despliega los enteros como números decimales. Note que %i es capaz de introducir enteros decimales, octales y hexadecimales.

Especificador de conversión	Descripción
Enteros	
d	Lee un entero decimal, opcionalmente signado. El argumento correspondiente es un apuntador a un entero.
i	Lee un entero decimal, octal o hexadecimal, opcionalmente signado. El argumento correspondiente es un apuntador a un entero.
o	Lee un entero octal. El argumento correspondiente es un apuntador a un entero no signado.
u	Lee un entero decimal no signado. El argumento correspondiente es un apuntador a un entero no signado.
x o X	Lee un entero hexadecimal. El argumento correspondiente es un apuntador a un entero no signado.
h o l	Se coloca antes de cualquiera de los especificadores de conversión de enteros, para indicar que un entero <code>short</code> o <code>long</code> será introducido.
Números de punto flotante	
e, E, f, g, o G	Lee un valor en punto flotante. El argumento correspondiente es un apuntador a una variable de punto flotante.
l o L	Se coloca delante de cualquier especificador de conversión de punto flotante para indicar que un valor <code>double</code> o <code>long double</code> será introducido.
Caracteres y cadenas.	
c	Lee un carácter. El argumento correspondiente es un apuntador a <code>char</code> y <code>NULL</code> (<code>'\0'</code>) no se agrega.
s	Lee una cadena. El argumento correspondiente es un apuntador a un arreglo del tipo <code>char</code> , que es lo suficiente extenso para contener la cadena y un carácter de terminación <code>NULL</code> (<code>'\0'</code>).
Conjunto de rastreo [scan characters]	Rastrea una cadena buscando un conjunto de caracteres almacenados en un arreglo.
Misceláneos	
p	Lee una dirección de apuntador producido de la misma forma que cuando una dirección es extraída con <code>%p</code> en un enunciado <code>printf</code> .
n	Almacena el número de caracteres introducidos hasta el momento en este <code>scanf</code> . El argumento correspondiente es un apuntador a entero.
%	Saltarse un signo de por ciento (%) en la entrada.

Fig. 9.17 Especificadores de conversión para `scanf`.

Al introducir números de punto flotante, cualquiera de los especificadores de conversión de punto flotante `e`, `E`, `f`, `g`, o `G` pueden ser utilizados. El programa de la figura 9.19 demuestra la lectura de tres números de punto flotante, con cada uno de los tres tipos de los especificadores flotantes de conversión y muestra todos los tres números mediante el especificador de conversión `f`. Note que la salida del programa confirma el hecho que los valores del punto flotante no son precisos —este hecho queda resaltado en el segundo valor impreso.

```
/* Reading integers */
#include <stdio.h>

main()
{
    int a, b, c, d, e, f, g;

    printf("Enter seven integers: ");
    scanf("%d%i%i%i%o%u%x", &a, &b, &c, &d, &e, &f, &g);
    printf("The input displayed as decimal integers is:\n");
    printf("%d %d %d %d %d %d %d\n", a, b, c, d, e, f, g);
    return 0;
}
```

```
Enter seven integers: -70 -70 070 0x70 70 70 70
The input displayed as decimal integers is:
-70 -70 56 112 56 70 112
```

Fig. 9.18 Cómo leer entradas con especificadores de conversión a enteros.

```
/* Reading floating-point numbers */
#include <stdio.h>

main()
{
    float a, b, c;

    printf("Enter three floating-point numbers: \n");
    scanf("%e%f%g", &a, &b, &c);
    printf("Here are the numbers entered in plain\n");
    printf("floating-point notation:\n");
    printf("%f %f %f\n", a, b, c);
    return 0;
}
```

```
Enter three floating-point numbers:
1.27987 1.27987e+03 3.38476e-06
Here are the numbers entered in plain
floating-point notation:
1.279870
1279.869995
0.000003
```

Fig. 9.19 Cómo leer entradas con especificadores de conversión de punto flotante.

Los caracteres y cadenas se introducen utilizando los especificadores de conversión `c` y `s`, respectivamente. El programa de la figura 9.20 solicita al usuario que introduzca una cadena. El programa introduce el primer carácter de la cadena utilizando `%c` y lo almacena en la variable de

carácter **x**, y a continuación introduce el resto de la cadena, utilizando **%s** y lo almacena en el arreglo de caracteres **y**.

Se puede introducir una secuencia de caracteres utilizando un *conjunto de rastreo*. Un conjunto de rastreo es un conjunto de caracteres encerrados en corchetes **[]**, y precedidos por un signo por ciento en la cadena de control de formato. Un conjunto de rastreo rastrea los caracteres en el flujo de entrada, buscando sólo aquellos caracteres que coincidan con los caracteres contenidos en el conjunto de rastreo. Cada vez que se encuentre un carácter, se almacena en el argumento correspondiente del conjunto de caracteres que es un apuntador a un arreglo de caracteres. El conjunto de rastreo deja de introducir caracteres cuando encuentra un carácter que no esté contenido en el conjunto de rastreo. Si el primer carácter en el flujo de entrada no coincide con un carácter en el conjunto de rastreo, sólo se almacenará en el arreglo el carácter nulo. El programa en la figura 9.21 utiliza un conjunto de rastreo **[aeiou]** para rastrear el flujo de entrada buscando vocales. Observe que se leerán las primeras siete letras de la entrada. La octava letra (**h**) no forma parte del conjunto de rastreo y, por lo tanto, el rastreo terminará.

El conjunto de rastreo también puede ser utilizado para rastrear buscando caracteres que no estén contenidos en el mismo, utilizando un *conjunto de rastreo invertido*. Para crear un conjunto de rastreo invertido, coloque un *acento circunflejo* (**^**) en los corchetes antes de los caracteres de rastreo. Esto hará que sean almacenados los caracteres que no aparecen en el conjunto de rastreo. Cuando se detecte un carácter contenido en el conjunto de rastreo invertido, se terminará la entrada. El programa en la figura 9.22 utiliza el conjunto de rastreo invertido **[^aeiou]** para buscar consonantes o más apropiadamente para buscar “no vocales”.

En la especificación de conversión **scanf** se puede utilizar un ancho de campo para leer un número específico de caracteres a partir del flujo de entrada. El programa de la figura 9.23 introduce una serie de dígitos consecutivos como un entero de dos dígitos y un entero que consiste de los dígitos restantes en el flujo de entrada.

```
/* Reading characters and strings */
#include <stdio.h>

main()
{
    char x, y[9];

    printf("Enter a string: ");
    scanf("%c%s", &x, y);

    printf("The input was:\n");
    printf("the character \"%c\" ", x);
    printf("and the string \"%s\"\n", y);
    return 0;
}
```

Enter a string: Sunday
The input was:
the character "S" and the string "unday"

Fig. 9.20 Cómo introducir caracteres y cadenas.

```
/* Using a scan set */
#include <stdio.h>

main()
{
    char z[9];

    printf("Enter string: ");
    scanf("%[aeiou]", z);
    printf("The input was \"%s\"\n", z);
    return 0;
}
```

Enter String: ooeeoahah
The input was "ooeooda"

Fig. 9.21 Cómo usar un conjunto de rastreo.

```
/* Using an inverted scan set */
#include <stdio.h>

main()
{
    char z[9];

    printf("Enter a string: ");
    scanf("%[^aeiou]", z);
    printf("The input was \"%s\"\n", z);
    return 0;
}
```

Enter a string: String
The input was "Str"

Fig. 9.22 Cómo utilizar un conjunto de rastreo invertido.

A menudo es necesario omitir ciertos caracteres del flujo de entrada. Por ejemplo, una fecha podía ser introducida como

7-9-91

Es necesario almacenar cada número de la fecha, pero los guiones que separan los números pueden ser descartados. A fin de eliminar caracteres innecesarios, inclúyalos en la cadena de control de formato de **scanf** (caracteres de espacio en blanco —como es el espacio en blanco, la nueva línea y el tabulador— pasar por alto todos los espacios en blanco a la izquierda). Por ejemplo, para pasar por alto los guiones en la entrada, utilice el enunciado

scanf("%d-%d-%d", &month, &day, &year);

```

/* inputting data with a field width */
#include <stdio.h>

main()
{
    int x, y;

    printf("Enter a six digit integer: ");
    scanf("%2d%d", &x, &y);
    printf("The integers input were %d and %d\n", x, y);
    return 0;
}

```

```

Enter a six digit integer: 123456
The integers input were 12 and 3456

```

Fig. 9.23 Cómo introducir datos con un ancho de campo.

Aunque este `scanf` si elimina los guiones de la entrada anterior, es posible que la fecha hubiera sido escrita como

```
7/9/91
```

En este caso el `scanf` precedente no eliminaría caracteres innecesarios. Por esta razón, `scanf` proporciona un *carácter de supresión de asignación* *. El carácter de supresión de asignación, le permite a `scanf` leer cualquier tipo de datos a partir de la entrada y descartarlos sin asignarlos a una variable. El programa de la figura 9.24 utiliza el carácter de supresión de asignación en la especificación de conversión `%c`, para indicar que un carácter que aparece en el flujo de entrada deberá ser leído y descartado. Sólo se almacenará mes, día y año. Los valores de las variables se imprimen para demostrar que de hecho han sido introducidos en forma correcta. Note que ninguna variable en la lista de argumentos corresponde a las especificaciones de conversión que utiliza el carácter de supresión de asignación, porque con estas especificaciones de conversión no se efectúa ninguna asignación.

Resumen

- Toda entrada y salida se maneja a través de flujos —secuencias de caracteres organizadas en línea. Cada línea consiste de cero o más caracteres y terminan con el carácter de nueva línea.
- Por lo regular, el flujo estándar de entrada se conecta al teclado, y el flujo estándar de salida está conectado a la pantalla de la computadora.
- Los sistemas operativos a menudo permiten que los flujos de entrada y salida estándar, sean redirigidos a otros dispositivos.
- La cadena de control de formato `printf` describe el formato o formatos en los cuales aparecerán los valores de salida. La cadena de control de formato está formada de los especificadores de conversión, las banderas, los anchos de campo, las precisiones y los caracteres literales.

```

/* Reading and discarding characters from the input stream */
#include <stdio.h>

main()
{
    int month1, day1, year1, month2, day2, year2;

    printf("Enter a date in the form mm-dd-yy: ");
    scanf("%d%c%d%c%d", &month1, &day1, &year1);
    printf("month = %d day = %d year = %d\n\n",
           month1, day1, year1);
    printf("Enter a date in the form mm/dd/yy: ");
    scanf("%d%c%d%c%d", &month2, &day2, &year2);
    printf("month = %d day = %d year = %d\n",
           month2, day2, year2);
    return 0;
}

```

```

Enter a date in the form mm-dd-yy: 11-18-71
month = 11 day = 18 year = 71

Enter a date in the form mm/dd/yy: 11/18/71
month = 11 day = 18 year = 71

```

Fig. 9.24 Cómo leer y descartar caracteres del flujo de entrada.

- Los enteros se imprimen con los especificadores de conversión siguientes : `d` o `i` para enteros opcionalmente signados, o para enteros no signados en forma octal, `u` para enteros no signados en forma decimal, y `x` o bien `X` para enteros no signados en forma hexadecimal. Para indicar un entero `short` o `long` respectivamente a los especificadores de conversión ya descritos, se antecede el modificador `h` o `l`.
- Se imprimen los valores en punto flotante utilizando los especificadores de conversión siguientes: `e` o `E` en el caso de notación exponencial, `f` para la notación de punto flotante normal y `g` o `G` para ya sea `e` o (`E`) o para la notación `f`. Cuando se indica el especificador de conversión `g` o (`G`), si el exponente del valor es menor de `-4` o mayor o igual que la precisión con la cual se imprime el valor, se utiliza el especificador de conversión `e` o (`E`).
- La precisión para los especificadores de conversión `g` y `G` indica el número máximo de dígitos significativos que se imprimirán.
- El especificador de conversión `c` imprime un carácter.
- El especificador de conversión `s` imprime una cadena de caracteres terminadas por un carácter nulo.
- El especificador de conversión `p` despliega una dirección de apuntador en forma de puesta en marcha definida (en muchos sistemas se utiliza la notación hexadecimal).
- El especificador de conversión `n` almacena el número de caracteres ya salido en el enunciado actual `printf`. El argumento correspondiente es un apuntador a un entero.

- El especificador de conversión **%%** hace que salga un **%** literal.
- Si el ancho de campo es mayor que el objeto que se está imprimiendo, el objeto quedará justificado a la derecha dentro de dicho campo.
- Los anchos de campo pueden ser utilizados por todos los especificadores de conversión.
- La precisión utilizada con los especificadores de conversión enteros indican el número mínimo de dígitos a imprimirse. Si el valor contiene menos dígitos que la precisión especificada, se antecederán ceros al valor impreso, hasta que el número de dígitos sea equivalente a la precisión.
- La precisión utilizada con especificadores de conversión de punto flotante **e**, **E**, y **f** indica el número de dígitos que aparecerán después del punto decimal.
- La precisión utilizada con especificadores de conversión de punto flotante **g** y **G** indica el número de dígitos significativos que aparecerán.
- La precisión utilizada con el especificador de conversión **s** indica el número de caracteres a imprimirse.
- El ancho de campo y la precisión pueden ser combinados, colocando el ancho de campo seguido por un punto decimal y a su vez seguido por la precisión, entre el **%** y el especificador de conversión.
- Es posible especificar el ancho de campo y la precisión mediante expresiones enteras en la lista de argumentos, a continuación de la cadena de control de formato. Para utilizar esta característica, inserte un ***** (asterisco) en lugar del ancho de campo o de la precisión. El argumento coincidente en la lista de argumentos se evalúa y se utilizará en lugar del asterisco. El valor del argumento puede ser negativo para el ancho de campo, pero para la precisión deberá ser positivo.
- La bandera signo de **-** justifica a la izquierda su argumento dentro de un campo.
- La bandera **+** imprime un signo más para valores positivos y un signo menos para valores negativos.
- La bandera de espacio imprime un espacio antes de un valor positivo, que no esté mostrado con la bandera de **+**.
- La bandera de signo de **#** antecede un **0** a los valores octal, **0x** o **0X** a los valores hexadecimales, y obliga a que se imprima el punto decimal en el caso de valores de punto flotante impresos con **e**, **E**, **f**, **g**, o **G** (normalmente el punto decimal se desplegará sólo si el valor contiene una parte fraccionaria).
- La bandera **0** imprime ceros a la izquierda para un valor que no ocupa todo su ancho de campo.
- Se consigue un formato preciso de entrada utilizando la función de biblioteca **scanf**.
- Los enteros se introducen mediante los especificadores de conversión **d** e **i** para enteros opcionalmente signados, y **o**, **u**, **x**, o **X** para enteros no signados. Para introducir un entero **short** o bien **long** respectivamente, se colocan los modificadores **h** y **l** antes de un especificador de conversión entero.
- Los valores de punto flotante son introducidos con los especificadores de conversión **e**, **E**, **f**, **g**, o **G**. Se colocan los modificadores **l** y **L** antes de cualquiera de los especificadores de conversión de punto flotante, para indicar que el valor introducido es un valor **double**, o bien **long double**, respectivamente.
- Los caracteres se introducen mediante el especificador de conversión **c**.
- Las cadenas se introducen mediante el especificador de conversión **s**.

- Un conjunto de rastreo, rastrea los caracteres en la entrada buscando sólo aquellos caracteres que coincidan con los caracteres que contiene el conjunto de rastreo. Cuando se encuentra un carácter, se almacena un arreglo de caracteres. El conjunto de rastreo deja de introducir caracteres cuando es encontrado un carácter no contenido en el conjunto de rastreo.
- Para crear un conjunto de rastreo invertido, coloque un acento circunflejo (**^**) en los corchetes, antes de los caracteres de rastreo. Esto hace que los caracteres que no aparecen en el conjunto de rastreo sean almacenados hasta que se encuentre un carácter contenido en el conjunto de rastreo invertido.
- Los valores de dirección se introducen utilizando el especificador de conversión **p**.
- El especificador de conversión **n** almacena el número de caracteres ya introducido en el **scanf** actual. El argumento correspondiente es un apuntador a **int**.
- La especificación de conversión **%%** hace coincidir en la entrada un carácter único de **%**.
- El carácter de supresión de asignación se utiliza para leer datos del flujo de entrada y descartar dichos datos.
- Un ancho de campo se puede utilizar en un **scanf** para leer un número específico de caracteres a partir de flujo de entrada.

Terminología

bandera **#**
especificador de conversión **%**
***** en ancho de campo
***** en precisión
bandera **+** (signo de más)
bandera **-** (signo de menos)
bandera **0** (cero)
<stdio.h>
secuencia de escape **\"**
secuencia de escape **\'**
secuencia de escape **\?**
secuencia de escape ****
secuencia de escape **\a**
secuencia de escape **\b**
secuencia de escape **\f**
secuencia de escape **\n**
secuencia de escape **\r**
secuencia de escape **\t**
secuencia de escape **\v**
alineación
carácter de supresión de asignación (*****)
inserción de espacio en blanco
especificador de conversión **c**
acento circunflejo (**^**)
especificación de conversión
especificadores de conversión
especificador de conversión **d**
especificador de conversión **e** o **E**
secuencia de escape

formato exponencial de punto flotante
especificador de conversión **f**
ancho de campo
bandera
punto flotante
cadena de control de formato
especificador de conversión **g** o **G**
especificador de conversión **h**
formato hexadecimal
especificador de conversión **l**
especificadores de conversión enteros
conjunto de rastreo invertido
especificador de conversión **L**
especificador de conversión **l**
justificación a la izquierda
caracteres literales
entero **long**
especificador de conversión **n**
especificador de conversión **o**
formato octal
especificador de conversión **p**
precisión
printf
inserción de carácter de impresión
redireccionamiento de un flujo
justificación a la derecha
redondeo
especificador de conversión **s**
conjunto de rastreo

scanf	flujo de salida estándar
notación científica	flujo
entero short	especificador de conversión u
formato entero signado	formato de entero no signado
bandera de espacio	espacio en blanco
flujo estándar de error	especificador de conversión x (o X)
flujo de entrada estándar	

Errores comunes de programación

- 9.1 Olvidar encerrar entre comillas una cadena de control de formato.
- 9.2 Imprimir un valor negativo utilizando un especificador de conversión que espera un valor no signado.
- 9.3 Utilizar **%c** para imprimir el primer carácter de una cadena. La especificación de conversión **%c** espera un argumento **char**. Una cadena es un apuntador a **char**, es decir, a **char ***.
- 9.4 Utilizar **%s** para imprimir un argumento **char**. La especificación de conversión **%s** espera un argumento de tipo apuntador a **char**. En algunos sistemas, esto causará un error fatal en tiempo de ejecución, conocido como violación de acceso.
- 9.5 Es un error de sintaxis usar comillas sencillas alrededor de cadenas de caracteres. Las cadenas de caracteres deben estar encerradas entre comillas dobles.
- 9.6 Usar comillas dobles alrededor de una constante de carácter. Esto, de hecho, crea una cadena formada por dos caracteres, el segundo de los cuales es el carácter de terminación **NULL**. Una constante de carácter es un carácter, encerrado entre comillas sencillas.
- 9.7 Intentar imprimir un carácter literal de por ciento, utilizando en la cadena de control de formato **%** en vez de **%%**. Cuando en la cadena de control de formato aparece el signo de **%**, debe ser seguida por un especificador de conversión.
- 9.8 No proporcionar un ancho de campo lo suficiente extenso para manejar un valor a imprimirse. Esto puede desplazar otros datos imprimiéndose y puede producir salidas confusas. ¡Familiarícese con sus datos!
- 9.9 Intentar imprimir como datos literales en un enunciado **printf** una comilla, dobles comillas, signo de interrogación o un carácter de diagonal invertida, sin anteceder dichos caracteres por una diagonal invertida para formar una secuencia de escape correcta.

Prácticas sanas de programación

- 9.1 Edite las salidas de manera nítida para su presentación. Esto hace más legible las salidas del programa y reduce errores de usuario.
- 9.2 Al extraer datos, asegúrese que el usuario está consciente de situaciones en las cuales, debido al formato, los datos pudieran resultar imprecisos (por ejemplo, errores de redondeo debidos a precisiones especificadas).
- 9.3 Al introducir datos, solicite al usuario un elemento o pocos elementos de datos a la vez. Evite solicitar al usuario la introducción de muchos elementos de datos, en respuesta a una sola solicitud.

Sugerencias de portabilidad

- 9.1 El especificador de conversión **p** despliega una dirección de apuntador en forma de puesta en marcha definida (en muchos sistemas se utiliza notación hexadecimal en preferencia a notación decimal).

Ejercicios de autoevaluación

- 9.1 Llene los espacios de cada uno de los siguientes:

- a) Todas las entradas y salidas se manejan en forma de _____.
 - b) El flujo de _____ se conecta por lo regular al teclado
 - c) El flujo de _____ se conecta por lo regular a la pantalla de la computadora.
 - d) Un formato preciso de salida se consigue con la función _____.
 - e) La cadena de control de formato puede contener _____, _____, _____, _____, y _____.
 - f) El especificador de conversión _____, o bien _____ puede ser utilizado para la salida de un entero decimal signado.
 - g) Los especificadores de conversión _____, _____, y _____ se utilizan para desplegar enteros no signados en forma octal, decimal y hexadecimal, respectivamente.
 - h) Se colocan los modificadores _____, y _____ antes de los especificadores de conversión enteros, a fin de indicar que se despliegan valores enteros **short**, o bien **long**.
 - i) El especificador de conversión _____ se utiliza para desplegar un valor en punto flotante en notación exponencial.
 - j) El modificador _____ se coloca antes de cualquier especificador de conversión de punto flotante para indicar que se va a delegar un valor **long double**.
 - k) Si no se ha especificado precisión, los especificadores de conversión **e**, **E** y **f** se despliegan con _____ dígitos de precisión a la derecha del punto decimal.
 - l) Para imprimir cadenas y caracteres respectivamente se utilizan los especificadores de conversión _____, y _____.
 - m) Todas las cadenas terminan con el carácter _____.
 - n) El ancho de campo y la precisión en una especificación de conversión **printf** pueden ser controladas con expresiones enteras, substituyendo un _____ en lugar del ancho de campo o de la precisión, y colocando la expresión entera en el argumento correspondiente de la lista de argumento.
 - o) La bandera _____ hace que en un campo la salida quede justificada a la izquierda.
 - p) La bandera _____ hace que se desplieguen valores, ya sea con un signo de más o con un signo de menos.
 - q) Se consigue un formato de entrada preciso con la función _____.
 - r) Se utiliza un _____ para rastrear una cadena buscando caracteres específicos y almacenar dichos caracteres en un arreglo.
 - s) El especificador de conversión _____ puede ser utilizado para introducir enteros octales, decimales y hexadecimales opcionalmente signados.
 - t) El especificador de conversión _____ puede ser utilizado para introducir un valor **double**.
 - u) El _____ se utiliza para leer datos del flujo de entrada y descartarlos sin asignarlos a una variable.
 - v) Se puede utilizar un _____ en una especificación de conversión **scanf** para indicar que un número específico de caracteres o de dígitos debe ser leído a partir del flujo de entrada.
- 9.2 Encuentre el error en cada uno de los siguientes y explique cómo puede ser corregido.
- a) El siguiente enunciado deberá imprimir el carácter 'c'

```
printf(" %s\n", 'c');
```
 - b) El siguiente enunciado debería imprimir 9.375%

```
printf("%.3f%", 9.375);
```
 - c) El siguiente enunciado debería imprimir el primer carácter de la cadena "Monday"

```
printf("%c\n", "Monday");
```
 - d)

```
printf("A string in quotes");
```
 - e)

```
printf(%de, 12, 20);
```
 - f)

```
printf("%c", "x");
```
 - g)

```
printf("%s\n", 'Richard');
```

9.3 Escriba un enunciado para cada uno de los siguientes:

- Imprima 1234, justificados a la derecha, en un campo de 10 dígitos.
- Imprima 123.456789, en notación exponencial, con un signo de (+ o de -) y 3 dígitos de precisión.
- Lea un valor `double` a la variable `number`.
- Imprima 100 en forma octal, precedida por 0.
- Lea una cadena del arreglo de caracteres `string`.
- Lea caracteres al arreglo `n`, hasta que se encuentre un carácter no dígito.
- Utilice las variables enteras `x` y `y`, para especificar un ancho de campo y una precisión utilizada para desplegar el valor `double` 87.4573.
- Lea un valor de la forma 3.5%. Almacene el porcentaje en la variable `float percent`, y elimine el signo de % del flujo de entrada. No utilice el carácter de supresión de asignación.
- Imprima 3.333333 como un valor `long double` con un signo (+ o -) en un campo de 20 caracteres con una precisión de 3.

Respuestas a los ejercicios de autoevaluación

9.1 a) Flujos. b) Entrada estándar. c) Salida estándar. d) `printf`. e) Especificadores de conversión, bandera, anchos de campo, precisiones y caracteres literales. f) `d`, `i`. g) `o`, `u`, `x` (o `X`). h) `h`, `l`. i) `e` (o `E`). j) `L`, `k`. l) `s`, `c`. m) `NULL` ('`\0`'). n) asterisco (*). o) - (menos). p) + (más). q) `scanf`. r) Conjunto de rastreo. s) `l`, `t`. t) `le`, `1E`, `1f`, `1g`, o bien `1G`. u) Carácter de supresión de asignación (*). v) Ancho de campo.

- 9.2 a) Error: el especificador de conversión `s` espera un argumento del tipo apuntador a `char`. Corrección: para imprimir el carácter '`c`', utilice la especificación de conversión `%c`, o bien cambie '`c`' a '`%c`'.
- b) Intentar imprimir el carácter literal `%` sin utilizar la especificador de conversión `%%`. Corrección: Utilice `%%` para imprimir un carácter literal `%`.
- c) Error: el especificador de conversión `c` espera un argumento del tipo `char`. Corrección: Para imprimir el primer carácter de "`Monday`" utilice el especificador de conversión `%ls`.
- d) Error: tratar de imprimir el carácter literal `"`, sin utilizar la secuencia de escape `\`. Corrección: Remplace cada comilla en el conjunto interior de comillas, con `\`.
- e) Error: la cadena de control de formato no está encerrada entre comillas dobles. Corrección: Encierre `%d%d` entre comillas dobles.
- f) El carácter `x` está encerrado entre comillas dobles. Corrección: los constantes de caracteres a ser impresos utilizando `%c` deben estar encerrados entre comillas sencillas.
- g) Error: la cadena a imprimirse está encerrada entre comillas sencillas. Corrección: utilice comillas dobles en vez de sencillas para representar a una cadena.

- 9.3 a) `printf("%10d\n", 1234);`
 b) `printf("%+.3e\n", 123.456789);`
 c) `scanf("%lf", &number);`
 d) `printf("%#o\n", 100);`
 e) `scanf("%s", string);`
 f) `scanf("[%0123456789]", n);`
 g) `printf("%*.f\n", x, y, 87.4573);`
 h) `scanf("%f%%", &percent);`
 i) `printf("%+20.3Lf\n", 3.333333);`

Ejercicios

9.4 Escriba un enunciado `printf` o bien `scanf` para cada uno de los siguientes:

- Imprima el entero no signado 40000 justificado a la izquierda, en un campo de 15 dígitos, con 8 dígitos.
- Lea un valor hexadecimal a la variable `hex`.
- Imprima 200 con y sin un signo.
- Imprima 100 en forma hexadecimal precedido por `0x`.
- Lea caracteres al arreglo `s`, hasta que se encuentre con la letra `p`.
- Imprima 1.234 en un campo de 9 dígitos, con ceros a la izquierda.
- Lea una hora de la forma `hh:mm:ss` almacenando las partes de la hora en las variables enteras `hour`, `minute` y `second`. Omita los dobles puntos (:) del flujo de entrada. Utilice el carácter de supresión de asignación.
- Lea una cadena de la forma "`characters`" de la entrada estándar. Almacene la cadena en el arreglo de carácter `s`. Elimine las comillas del flujo de entrada.
- Lea una hora de la forma `hh:mm:ss` almacenando las partes de la hora en las variables enteras `hour`, `minute` y `second`. Omita los dobles puntos (:) en el flujo de entrada. No utilice el carácter de supresión de asignación.

9.5 Muestre lo que se imprime mediante cada uno de los enunciados siguientes. Si un enunciado es incorrecto, indique porqué.

- `printf("%-10d\n", 10000);`
- `printf("%c\n", "This is a string");`
- `printf("%*.1f\n", 8, 3, 1024.987654);`
- `printf("%#o\n%X\n#e\n", 17, 17, 1008.83689);`
- `printf("%ld\n%ld\n", 1000000, 1000000);`
- `printf("%10.2E\n", 444.93738);`
- `printf("%10.2g\n", 444.93738);`
- `printf("%d\n", 10.987);`

9.6 Encuentre el o los errores en cada uno de los siguientes segmentos de programa. Explique cómo pueden ser corregidos cada uno de ellos.

- `printf("%s\n", 'Happy Birthday');`
- `printf("%c\n", 'Hello');`
- `printf("%c\n", "This is a string");`
- El siguiente enunciado deberá imprimir "`Bon Voyage`"
`printf("%s", "Bon Voyage");`
- `char day [] = "Sunday";`
`printf("%s\n", day [3]);`
- `printf('Enter your name: ');`
- `printf("%f", 123.456);`
- El siguiente enunciado debería de imprimir los caracteres '`O`' y '`K`'.
`printf("%s%s\n", 'O', 'K');`
- `char s[10];`
`scanf("%c", s[7]);`

9.7 Escriba un programa que cargue el arreglo de 10 elementos `number` con enteros al azar, desde 1 hasta 1000. Para cada uno de los valores, imprima el valor y el total acumulado del número de caracteres impresos. Utilice la especificación de conversión `%n` para determinar el número de caracteres ya extraídos para cada valor. Imprima el número total de caracteres extraídos para todos los valores incluyendo el valor actual, cada vez que éste sea impreso. La salida deberá tener el formato siguiente:

Value	Total characters
342	3
1000	7
963	10
6	11
etc.	

9.8 Escriba un programa para probar la diferencia entre los especificadores de conversión %d y %i al ser utilizados en enunciados `scanf`. Utilice los enunciados

```
scanf("%i%d", &x, &y);
printf("%d%d\n", x, y);
```

para introducir e imprimir los valores. Pruebe el programa con los siguientes conjuntos de datos de entrada:

```
10      10
-10     -10
010     010
0x10    0x10
```

9.9 Escriba un programa que imprima los valores de apuntador utilizando todos los especificadores de conversión enteros y la especificación de conversión %p. ¿Cuáles son los que imprimen valores raros? ¿Cuáles son los que causan errores? ¿En cuál de los formatos la especificación de conversión %p despliega en su sistema la dirección?

9.10 Escriba un programa para probar el resultado de imprimir el valor entero 12345 y el de punto flotante 1.2345 en varios tamaños de campo. ¿Qué pasa cuando se imprimen los valores en campos que contienen menos dígitos que los valores mismos?

9.11 Escriba un programa que imprima el valor 100.453627 redondeado al dígito, décima, centésima, milésima y decenas de millar más cercano.

9.12 Escriba un programa que desde el teclado introduzca una cadena y determine la longitud de la misma. Imprima la cadena utilizando como ancho de campo dos veces su longitud.

9.13 Escriba un programa que convierta temperaturas Fahrenheit enteras desde 0 hasta 212 grados a temperaturas Celsius de punto flotante con 3 dígitos de precisión. Utilice la fórmula

```
celsius = 5.0 / 9.0 * (fahrenheit - 32);
```

para llevar a cabo el cálculo. La salida deberá ser impresa en dos columnas justificadas a la derecha, cada una de 10 caracteres, y las temperaturas Celsius deberán ser anteceditas por un signo, tanto para valores positivos como negativos.

9.14 Escriba un programa para probar todas las secuencias de escape de la figura 9.16. Para aquellas secuencias de escape que mueven el cursor, imprima un carácter, antes y después de imprimir la secuencia de escape, a fin de que resulte claro dónde se ha movido el cursor.

9.15 Escriba un programa que determine si ? puede ser impreso como un carácter literal como parte de una cadena de control de formato `printf`, en vez de utilizar la secuencia de escape \?

9.16 Escriba un programa que introduzca el valor 437 utilizando cada uno de los especificadores de conversión enteros `scanf`. Imprima cada valor introducido, utilizando todos los especificadores de conversión enteros.

9.17 Escriba un programa que utilice cada uno de los especificadores de conversión e, f y g para introducir el valor 1.2345. Imprima los valores de cada variable para probar que cada especificador de conversión puede ser utilizado para introducir este mismo valor.

9.18 En algunos lenguajes de programación, las cadenas pueden ser introducidas entre comillas sencillas o dobles. Escriba un programa que lea las tres cadenas `suzy`, `"suzy"`, y `'suzy'`. ¿Son las comillas sencillas y dobles ignoradas o leídas por C, como parte de la cadena?

9.19 Escriba un programa que determine si ? puede ser impreso como la constante de carácter '?', en vez de la secuencia de escape de carácter constante '\?' mediante el uso de especificador de conversión %c en la cadena de control de formato de un enunciado `printf`.

9.20 Escriba un programa que utilice el especificador de conversión g para extraer el valor 9876.12345. Imprima el valor con precisiones que vayan desde 1 hasta 9.

10

Estructuras, uniones, manipulaciones de bits y enumeraciones

Objetivos

- Ser capaz de crear y utilizar estructuras, uniones y enumeraciones.
- Ser capaz de pasar estructuras a funciones en llamada por valor y en llamada por referencia.
- Ser capaz de manipular datos con operadores a nivel de bits.
- Ser capaz de crear campos de bits para almacenar datos en forma compacta.

Nunca pude comprender lo que esos malditos puntos significaban.
Winston Churchill

Pero otra vez una unión en desunión;
William Shakespeare

Puedes excluirme.
Samuel Goldwyn

*La misma caritativa y vieja mentira
Repetida conforme los años pasan
Siempre con el mismo éxito—
“¡Realmente no has cambiado nada!”*
Margaret Fishback

Sinopsis

- 10.1 Introducción
- 10.2 Definiciones de estructuras
- 10.3 Cómo inicializar estructuras
- 10.4 Cómo tener acceso a miembros de estructuras
- 10.5 Cómo utilizar estructuras con funciones
- 10.6 Typedef
- 10.7 Ejemplo: simulación de barajar y distribuir cartas de alto rendimiento.
- 10.8 Uniones
- 10.9 Operadores a nivel de bitss
- 10.10 Campos de bitss
- 10.11 Constantes de enumeración

Resumen • Terminología • Errores comunes de programación • Prácticas sanas de programación • Sugerencias de portabilidad • Sugerencias de rendimiento • Observación de ingeniería de software • Ejercicios de autoevaluación • Respuestas a los ejercicios de autoevaluación • Ejercicios.

10.1 introducción

Las *estructuras* son colecciones de variables relacionadas —a veces denominadas *agregados*— bajo un nombre. Las estructuras pueden contener variables de muchos tipos diferentes de datos —a diferencia de los arreglos, que contienen únicamente elementos de un mismo tipo de datos. Generalmente las estructuras se utilizan para definir registros a almacenarse en archivos (vea el capítulo 11, “Procesamiento de archivos”). Los apuntadores y las estructuras facilitan la formación de estructuras de datos de mayor complejidad, como son listas enlazadas, colas de espera, pilas y árboles (vea el capítulo 12, “Estructuras de datos”).

10.2 Definiciones de estructuras

Las estructuras son *tipos de datos derivados* —están construidas utilizando objetos de otros tipos. Considere la siguiente definición de estructura:

```
struct card {
    char *face;
    char *suit;
};
```

La palabra reservada **struct** presenta la definición de estructura. El identificador **card** es el *rótulo de la estructura*. El rótulo de la estructura da nombre a la definición de la misma, y se utiliza con la palabra reservada **struct** para declarar variables del *tipo estructura*. En este ejemplo, el tipo estructura es **struct card**. Las variables declaradas dentro de las llaves de la definición de estructura son los *miembros* de la estructura. Los miembros de la misma estructura

deben tener nombres únicos, pero dos estructuras diferentes pueden contener miembros con el mismo nombre sin entrar en conflicto (pronto veremos por qué). Cada definición de estructura debe terminar con un punto y coma.

Error común de programación 10.1

Olvidar el punto y coma que da por terminada una definición de estructura.

La definición de **struct card** contiene dos miembros del tipo **char *** —**face** y **suit**. Los miembros de la estructura pueden ser variables de los tipos de datos básicos (es decir, **int**, **float**, etcétera), o agregados, como son los arreglos y otras estructuras. Como ya vimos en el capítulo 6, cada elemento de un arreglo debe ser del mismo tipo. Los miembros de una estructura, sin embargo, pueden ser de una variedad de tipos de datos. Por ejemplo, un **struct employee** pudiera contener miembros de cadenas de caracteres correspondientes a los nombres y apellidos, un miembro **int**, para la edad del empleado, un miembro **char**, que contenga ‘M’ o bien ‘F’ para el sexo del empleado, un miembro **float** para el salario horario del empleado, y así sucesivamente. Una estructura no puede contener una instancia de sí misma. Por ejemplo, una variable del tipo **struct card** no puede ser declarada dentro de la definición correspondiente a **struct card**. Sin embargo, pudiera ser incluido un apuntador a **struct card**. Una estructura, que contenga un miembro que es un apuntador al mismo tipo de estructura, se conoce como una *estructura autorreferenciada*. Las estructuras autorreferenciadas se utilizan en el capítulo 12 para construir varios tipos de estructuras de datos enlazadas.

La anterior definición de estructura no reserva ningún espacio en memoria, más bien genera un nuevo tipo de datos, que se utiliza para declarar variables. Las variables de estructura se declaran como se declaran las variables de otros tipos. La declaración

```
struct card a, deck[52], *c;
```

declara **a** ser una variable del tipo **struct card**, declara **deck** como un arreglo con 52 elementos del tipo **struct card**, y declara **c** como un apuntador a **struct card**. Las variables de un tipo dado de estructura, pudieran también ser declaradas colocando una lista, separada por comas, de los nombres de las variables, entre la llave de cierre de la definición de la estructura y el punto y coma que termina la definición de la misma. Por ejemplo, la declaración anterior podía haberse incorporado en la definición de estructura **struct card** como sigue:

```
struct card {
    char *face;
    char *suit;
} a, deck[52], *c;
```

El nombre del rótulo de la estructura es opcional. Si la definición de una estructura no contiene un nombre de rótulo de estructura, las variables de ese tipo de estructura pueden únicamente ser declaradas dentro de la definición de estructura —y no en una declaración por separado.

Práctica sana de programación 10.1

Al crear un tipo de estructura proporcione un nombre de rótulo de estructura. El nombre de rótulo de estructura es conveniente más adelante en el programa para la declaración de nuevas variables de este tipo de estructura.

Práctica sana de programación 10.2

Seleccionar un nombre de rótulo de estructura significativo ayuda a autodocumentar el programa.

Las únicas operaciones válidas que pueden ejecutarse sobre estructuras son: asignar variables de estructura a variables de estructura del mismo tipo, tomando la dirección (&) de una variable de estructura, obteniendo acceso a los miembros de una variable de estructura (vea la Sección 10.4), y utilizando el operador `sizeof`, a fin de determinar el tamaño de la variable de estructura.

Error común de programación 10.2

Asignar una estructura de un tipo a una estructura de un tipo distinto.

Las estructuras no pueden compararse entre sí, porque los miembros de las estructuras no están necesariamente almacenados en bytes de memoria consecutivos. Algunas veces en una estructura existen “huecos” porque las computadoras pudieran almacenar tipos de datos específicos en ciertos límites de memoria, como son límites de media palabra, de palabra o de dobles palabras. Una palabra es una unidad estándar de memoria, utilizada para almacenar datos en una computadora — 2 o 4 bytes, normalmente. Considere la siguiente definición de estructura, en la cual se declaran `sample1` y `sample2`, del tipo `struct example`:

```
struct example {
    char c;
    int i;
} sample1, sample2;
```

Una computadora con palabras de 2 bytes pudiera requerir que cada uno de los miembros de `struct example` fuesen alineados en un límite de palabras, es decir, al principio de una palabra (esto depende de la máquina). En la Figura 10.1 se muestra una alineación de almacenamiento para una variable del tipo `struct example`, que ha sido asignado al carácter ‘a’, y el entero 97 (se muestran las representaciones de los valores en bits). Si los miembros se almacenan empezando en los límites de palabras, aparece un hueco de 1 byte (byte 1 en la figura) en el almacenamiento para variables del tipo `struct example`. El valor en el hueco de un byte se queda sin definir. Si los valores de miembros de `sample1` y `sample2` son de hecho iguales, la comparación de las estructuras no será necesariamente igual, porque los huecos no definidos de un byte probablemente no contendrán valores idénticos.

Error común de programación 10.3

Es un error de sintaxis comparar estructuras, debido a diferentes requisitos de alineación en los diferentes sistemas.

Sugerencia de portabilidad 10.1

Dado que depende de la máquina el tamaño de los elementos de datos de un tipo particular, y debido a que las consideraciones de alineación de almacenamiento también son dependientes de la máquina, entonces también lo será la representación de una estructura.

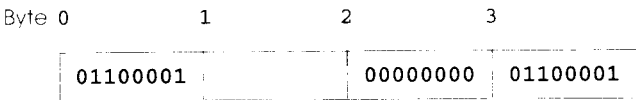


Fig. 10.1 Una posible alineación de almacenamiento para una variable del tipo `struct example` mostrando en la memoria un área no definida.

10.3 Cómo inicializar estructuras

Las estructuras pueden ser inicializadas mediante listas de inicialización como con los arreglos. Para inicializar una estructura, escriba en la declaración de la estructura, a continuación del nombre de la variable, un signo igual, con inicializadores encerrados entre llaves y separados por comas. Por ejemplo, la declaración

```
struct card a = ("Three", "Hearts");
```

crea la variable `a` del tipo `struct card` (como fue definida anteriormente) e inicializa el miembro `face` a `"Three"`, y el miembro `suit` a `"Hearts"`. Si en la lista aparecen menos inicializadores que en la estructura, los miembros restantes automáticamente quedarán inicializados a 0 (o `NULL` si el miembro es un apuntador). Las variables de estructura declarados por fuera de una definición de función (es decir, en forma externa) se inicializan a 0 o `NULL` si en la declaración externa no se inicializan en forma explícita. Las variables de estructura también pueden ser inicializadas en enunciados de asignación, asignándoles una variable de estructura del mismo tipo, o asignando valores a los miembros individuales de la misma.

10.4 Cómo tener acceso a los miembros de estructuras

Para tener acceso a miembros de estructuras se utilizan dos operadores: el *operador de miembro de estructura* (`.`) —también conocido como *operador punto*— y el *operador de apuntador de estructura* (`->`) —también conocido como el *operador de flecha*. El operador de miembro de estructura tiene acceso a un miembro de estructura mediante el nombre de la variable de estructura. Por ejemplo, para imprimir el miembro `suit` de la estructura `a` correspondiente a la declaración anterior, utilice el enunciado

```
printf ("%s", a.suit);
```

El operador de apuntador de estructura —que consiste de un signo menos (`-`) y de un signo mayor que (`>`), sin espacios intermedios— tiene acceso a un miembro de estructura vía un apuntador a la estructura. Suponga que el apuntador `aPtr` se ha declarado para apuntar a `struct card`, y que la dirección de la estructura `a` ha sido asignada a `aPtr`. Para imprimir el miembro `suit` de la estructura `a` utilizando el apuntador `aPtr`, utilice el enunciado

```
printf ("%s", aPtr->suit);
```

La expresión `aPtr->suit` es equivalente `(*aPtr).suit` que desreferencia el apuntador y tiene acceso al miembro `suit` utilizando el operador de miembro de estructura. Se requiere aquí de los paréntesis, porque el operador de miembro de estructura (`.`) tiene una precedencia mayor que el operador de desreferenciación de apuntador (`*`). El operador de apuntador de estructura y el operador de miembro de estructura, junto con los paréntesis y los corchetes (`[]`) utilizados para los subíndices de arreglos, tienen la precedencia de operadores más alta y se asocian de izquierda a derecha.

Práctica sana de programación 10.3

Evite utilizar los mismos nombres para miembros de estructura de distintos tipos. Ello es permitido, pero podría causar confusión.

Práctica sana de programación 10.4

No deje espacios alrededor de los operadores `->` y `.` ya que ayuda a enfatizar que las expresiones en las cuales los operadores están contenidos son esencialmente nombres individuales de variables.

Error común de programación 10.4

Insertar un espacio entre el signo de `-` y el signo de `>` del operador de apuntador de estructura, (o insertar espacios entre los componentes de cualquier otro operador múltiple de teclado, a excepción de `?:`).

Error común de programación 10.5

Intentar referirse a un miembro de una estructura utilizando únicamente el nombre de dicho miembro.

Error común de programación 10.6

No utilizar paréntesis al referirse a un miembro de estructura utilizando un apuntador y el operador de miembro de estructura (por ejemplo `*aPtr.suit`, es un error de sintaxis).

El programa de la figura 10.2 pone de manifiesto el uso de los operadores de miembro de estructura y de apuntador de estructura. Mediante el uso del operador de miembro de estructura, los miembros de la estructura `a` son asignados los valores "Ace" y "Spades" respectivamente. Al apuntador `aPtr` se le asigna la dirección de la estructura `a`. Un enunciado `printf` imprime los miembros de la variable de estructura `a`, utilizando el operador de miembro de estructura con el nombre de variable `a`, el operador de apuntador de estructura con el apuntador `aPtr`, y el operador de miembro de estructura con el apuntador desreferenciado `aPtr`.

```
/* Using the structure member and
   structure pointer operators */
#include <stdio.h>

struct card {
    char *face;
    char *suit;
};

main()
{
    struct card a;
    struct card *aPtr;

    a.face = "Ace";
    a.suit = "Spades";
    aPtr = &a;
    printf("%s%s\n%s%s\n%s%s\n",
        a.face, " of ", a.suit,
        aPtr->face, " of ", aPtr->suit,
        (*aPtr).face, " of ", (*aPtr).suit);
    return 0;
}
```

```
Ace of Spades
Ace of Spades
Ace of Spades
```

Fig. 10.2 Cómo utilizar el operador de miembro de estructura y el operador de apuntador de estructura.

10.5 Cómo utilizar estructuras con funciones

Las estructuras pueden ser pasadas a funciones pasando miembros de estructura individuales, pasando toda la estructura, o pasando un apuntador a una estructura. Cuando se pasan estructuras o miembros individuales de estructura a una función, se pasan en llamada por valor. Por lo tanto, los miembros de la estructura de un llamador no podrán ser modificados por la función llamada.

Para pasar una estructura en llamada por referencia, pase la dirección de la variable de estructura. Los arreglos de estructura —como todos los demás otros arreglos— son automáticamente pasados en llamada por referencia.

En el capítulo 6, indicamos que un arreglo podía ser pasado en llamada por valor mediante el uso de una estructura. Para pasar un arreglo en llamada por valor, origine una estructura con el arreglo como un miembro. Dado que las estructuras se pasan en llamada por valor, el arreglo será pasado en llamada por valor.

Error común de programación 10.7

Suponer que las estructuras, como los arreglos, se pasan automáticamente en llamada por referencia, e intentar modificar los valores de estructura del llamador en la función llamada.

Sugerencia de rendimiento 10.1

Es más eficaz pasar estructuras en llamada por referencia que pasar estructuras en llamada por valor (ya que esto último requiere que toda la estructura se copie).

10.6 Typedef

La palabra reservada **typedef** proporciona un mecanismo para la creación de sinónimos (o alias) para tipos de datos anteriormente definidos. Los nombres de los tipos de estructura se definen a menudo utilizando **typedef**, a fin de crear nombres de tipo más breves. Por ejemplo, el enunciado

```
typedef struct card Card;
```

define el nuevo nombre de tipo `Card` como un sinónimo para el tipo `struct card`. Los programadores en C utilizan a menudo **typedef** para definir un tipo de estructura de tal forma que un rótulo de estructura no sea requerido. Por ejemplo, la definición siguiente

```
typedef struct {
    char *face;
    char *suit;
} Card;
```

crea el tipo de estructura `Card`, sin necesidad de un enunciado por separado **typedef**.

Práctica sana de programación 10.5

Ponga los nombres **typedef** en mayúsculas, para enfatizar que esos nombres son sinónimos de otros nombres de tipo.

`Card` puede ahora ser utilizado para declarar variables del tipo `struct card`. La declaración

```
Card deck[52];
```

declara un arreglo de 52 estructuras `Card` (es decir, variables del tipo `struct card`). Al crear un nuevo nombre utilizando **typedef** no se crea un nuevo tipo; **typedef** simplemente crea un

nuevo nombre de tipo, que puede ser utilizado como un seudónimo para un nombre de tipo existente. Un nombre significativo auxilia a autodocumentar el programa. Por ejemplo, cuando leemos la declaración anterior, sabemos que "deck es un arreglo de 52 Cards"

typedef se utiliza a menudo para crear seudónimos para los tipos de datos básicos. Por ejemplo, un programa que requiera de enteros de 4 bytes, pudiera utilizar el tipo **int** en un sistema y el tipo **long** en otro. Los programas diseñados para portabilidad, a menudo utilizan **typedef** para crear un alias o seudónimo para los enteros de 4 bytes como sería **Integer**. Una vez dentro del programa el alias **Integer** puede ser modificado, para hacer que el programa funcione en ambos sistemas.

Sugerencia de portabilidad 10.2

Utilice **typedef** para ayudar a hacer más portátil un programa.

10.7 Ejemplo: simulación de barajar y distribuir cartas de alto rendimiento

El programa en la figura 10.3 se basa en la simulación de barajar y distribuir cartas analizado en el capítulo 7. El programa representa el mazo de cartas o de naipes como un arreglo de estructuras. El programa utiliza algoritmos de alto rendimiento para barajar y distribuir. La salida del programa de alto rendimiento para barajar y distribuir se muestra en la figura 10.4.

En el programa, la función **fillDeck** inicializa el arreglo **Card** en orden desde Ace hasta King de cada uno de los palos. El arreglo **Card** se pasa a la función **shuffle**, donde se pone en operación el algoritmo de alto rendimiento de barajar. La función **shuffle** toma como argumento un arreglo de 52 estructuras **Card**. La función ciela a través de las 52 cartas (subíndices de arreglo 0 a 51) mediante una estructura **for**. Para cada una de las cartas, es tomado al azar un número entre 0 y 51. A continuación, en el arreglo son intercambiadas la estructura actual **Card** y la estructura seleccionada al azar **Card**. En una sola pasada de todo el arreglo se llevan a cabo un total de 52 intercambios, y ¡el arreglo de estructuras **Card** queda barajado! Este algoritmo no puede sufrir por posposición indefinida, como sufría el algoritmo de barajar presentado en el capítulo 7. Dado que en el arreglo las estructuras **Card** fueron intercambiadas en su lugar, el algoritmo de distribución de alto rendimiento puesto en marcha en la función **deal** requerirá de únicamente una pasada del arreglo para distribuir las cartas barajadas.

Error común de programación 10.8

Olvidar incluir el subíndice de arreglo al referirse a estructuras individuales de un arreglo de estructuras.

10.8 Uniones

Una **unión** es un tipo de datos derivado —como lo es una estructura— cuyos miembros comparten el mismo espacio de almacenamiento. Para distintas situaciones en un programa, algunas variables pudieran no ser de importancia, pero otras variables lo son —por lo que una unión comparte el espacio, en vez de desperdiciar almacenamiento en variables que no están siendo utilizadas. Los miembros de una unión pueden ser de cualquier tipo. El número de bytes utilizados para almacenar una unión, deben ser por lo menos suficientes para contener el miembro más grande. En la mayor parte de los casos, las uniones contienen dos o más tipos de datos. Únicamente un miembro y, por lo tanto, únicamente un tipo de datos, puede ser referenciado en un momento dado. Es responsabilidad del programador asegurarse que en una unión los datos están referenciados con el tipo de dato apropiado.

```
/* The card shuffling and dealing program using structures */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

struct card {
    char *face;
    char *suit;
};

typedef struct card Card;

void fillDeck(Card *, char *[], char *[]);
void shuffle(Card *);
void deal(Card *);

main()
{
    Card deck[52];
    char *face[] = {"Ace", "Deuce", "Three", "Four", "Five",
                    "Six", "Seven", "Eight", "Nine", "Ten",
                    "Jack", "Queen", "King"};
    char *suit[] = {"Hearts", "Diamonds", "Clubs", "Spades"};

    srand(time(NULL));

    fillDeck(deck, face, suit);
    shuffle(deck);
    deal(deck);
    return 0;
}

void fillDeck(Card *wDeck, char *wFace[], char *wSuit[])
{
    int i;

    for (i = 0; i <= 51; i++) {
        wDeck[i].face = wFace[i % 13];
        wDeck[i].suit = wSuit[i / 13];
    }
}

void shuffle(Card *wDeck)
{
    int i, j;
    Card temp;

    for (i = 0; i <= 51; i++) {
        j = rand() % 52;
        temp = wDeck[i];
        wDeck[i] = wDeck[j];
        wDeck[j] = temp;
    }
}
```

```
void deal(Card *wDeck)
{
    int i;

    for (i = 0; i < 52; i++)
        printf("%5s of %-8s%c", wDeck[i].face, wDeck[i].suit,
            (i + 1) % 2 ? '\t' : '\n');
}
```

Fig. 10.3 Simulación de barajar y distribuir cartas de alto rendimiento (parte 2 de 2).

Eight of Diamonds	Ace of Hearts
Eight of Clubs	Five of Spades
Seven of Hearts	Deuce of Diamonds
Ace of Clubs	Ten of Diamonds
Deuce of Spades	Six of Diamonds
Seven of Spades	Deuce of Clubs
Jacks of Clubs	Ten of Spades
King of Hearts	Jack of Diamonds
Three of Hearts	Three of Diamonds
Three of Clubs	Nine of Clubs
Ten of Hearts	Deuce of Hearts
Ten of Clubs	Seven of Diamonds
Six of Clubs	Queen of Spades
Six of Hearts	Three of Spades
Nine of Diamonds	Ace of Diamonds
Jack of Spades	Five of Clubs
King of Diamonds	Seven of Clubs
Nine of Spades	Four of Hearts
Six of Spades	Eight of Spades
Queen of Diamonds	Five of Diamonds
Ace of Spades	Nine of Hearts
King of Clubs	Five of Hearts
King of Spades	Four of Diamonds
Queen of Hearts	Eight of Hearts
Four of Spades	Jack of Hearts
Four of Clubs	Queen of Clubs

Fig. 10.4 Salida de la simulación de barajar y distribuir cartas de alto rendimiento.

Error común de programación 10.9

Es un error lógico referenciar con el tipo equivocado, datos en una unión almacenados con un tipo distinto.

Sugerencia de portabilidad 10.3

Si en una unión los datos se almacenan como de un tipo y se referencian como de otro tipo, los resultados serán dependientes de la instalación.

Una unión se declara con la palabra reservada **union** en el mismo formato que una estructura. La declaración **union**

```
union number{
    int x;
    float y;
};
```

indica que **number** es un tipo **union** con miembros **int x** y **float y**. En un programa normalmente la definición de unión antecede a **main**, por lo que ésta puede ser utilizada para declarar variables en todas las funciones del programa.

Observación de ingeniería de software 10.1

Al igual que en una declaración **struct**, una declaración **union** simplemente crea un tipo nuevo. Colocar una declaración **union** o **struct** fuera de cualquier función no crea una variable global.

Las operaciones que pueden ser ejecutadas en una unión son: asignar una unión a otra unión del mismo tipo, tomar la dirección (&) de una unión, y tener acceso a los miembros de una unión utilizando el operador de miembro de estructura y el operador de apuntador de estructura. Las uniones no pueden ser comparadas entre sí, por las mismas razones que no pueden compararse las estructuras.

En una declaración, una unión puede ser inicializada únicamente con un valor del mismo tipo que el primer miembro de la unión. Por ejemplo, en la unión anterior, la declaración

```
union number value = {10};
```

es una inicialización válida de la variable de unión **value**, porque la unión está inicializada con un **int**, pero la siguiente declaración no sería válida:

```
union number value = {1.43};
```

Error común de programación 10.10

Es un error de sintaxis comparar uniones, debido a los diferentes requisitos de alineación en varios sistemas.

Error común de programación 10.11

Inicialización de una uión en una declaración con un valor cuyo tipo es distinto al del primer miembro de la unión.

Sugerencia de portabilidad 10.4

La cantidad de almacenamiento requerido para almacenar una unión es dependiente de la instalación.

Sugerencia de portabilidad 10.5

Quizá no sea fácil portar algunas uniones a otros sistemas de computación. El que una unión sea portable o no depende a menudo de los requerimientos de alineación de almacenamiento para los tipos de miembro de unión de información en un sistema particular.

Sugerencia de rendimiento 10.2

Las uniones ahorran almacenamiento.

El programa de la figura 10.5 utiliza la variable `value` del tipo `union number`, para desplegar el valor almacenado en la unión, tanto como un `int` como como un `float`. La salida del programa depende de la instalación. La salida del programa muestra que la representación interna de un valor `float` puede resultar bastante distinta de la representación de `int`.

10.9 Operadores a nivel de bits

En las computadoras en forma interna todos los datos se representan como secuencias de bits. Cada bit puede asumir un valor de 0 o un valor de 1. En la mayor parte de los sistemas, una secuencia de 8 bits forma un byte —la unidad estándar de almacenamiento para una variable del tipo `char`. Otros tipos de datos son almacenados en números de bits más grandes. Los operadores

```
/* An example of a union */
#include <stdio.h>

union number {
    int x;
    float y;
};

main()
{
    union number value;

    value.x = 100;
    printf("%s\n%s\n%s%d\n%s%f\n\n",
        "Put a value in the integer member",
        "and print both members.",
        "int:  ", value.x,
        "float: ", value.y);

    value.y = 100.0;
    printf("%s\n%s\n%s%d\n%s%f\n",
        "Put a value in the floating member",
        "and print both members.",
        "int:  ", value.x,
        "float: ", value.y);

    return 0;
}
```

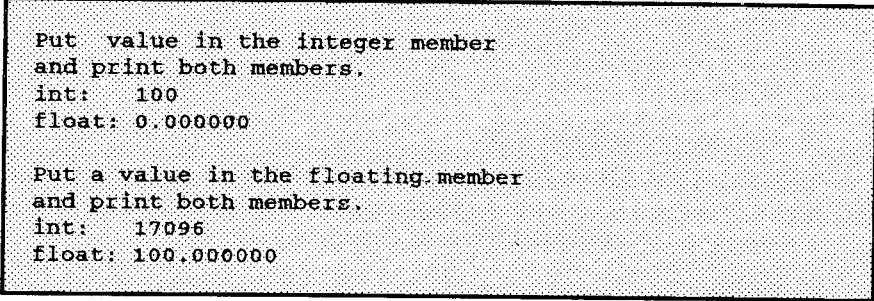


Fig. 10.5 Cómo imprimir el valor de una unión en ambos tipos de datos de miembro.

a nivel de bits se utilizan para manipular los bits de operandos integrales (`char`, `short`, `int` y `long`; tanto `signed` como `unsigned`). Los enteros no signados (`unsigned`) son utilizados normalmente con los operadores a nivel de bits.

Sugerencia de portabilidad 10.6

Las manipulaciones de datos a nivel de bits son dependientes de la máquina.

Advierta que los análisis de los operadores a nivel de bits de esta sección, muestran las representaciones binarias de los operandos enteros. Para una explicación detallada del sistema numérico binario (también conocido como de base 2) vea el Apéndice E, "Sistemas numéricos". También, los programas de las Secciones 10.9 y 10.10 fueron probados en un Macintosh de Apple, utilizando Think C y en una PC compatible, utilizando Borland C++. Ambos sistemas utilizan enteros de 16 bits (2 bytes). Dada la naturaleza de dependencia de la máquina de las manipulaciones a nivel de bits, estos programas pudieran no funcionar en su sistema.

Los operadores a nivel de bits son: *AND a nivel de bits* (`&`), *OR inclusivo a nivel de bits* (`|`), *OR exclusivo a nivel de bits* (`^`), *desplazamiento a la izquierda* (`<<`), *desplazamiento a la derecha* (`>>`), y *complemento* (`~`). El AND a nivel de bits, el OR inclusivo a nivel de bits y el OR exclusivo a nivel de bits son operadores que comparan sus dos operandos bit por bit. El operador AND a nivel de bits establece en el resultado cada bit a 1, si el bit correspondiente en ambos operandos es 1. El operador OR inclusivo a nivel de bits, establece en el resultado cada bit a 1, si el bit correspondiente en cada o en (ambos) operandos es 1. El operador OR exclusivo a nivel de bits establece en el resultado cada bit a 1 si el bit correspondiente en exactamente un operando es 1. El operador de desplazamiento a la izquierda desplaza los bits de su operando izquierdo hacia la izquierda por el número de bits especificado en su operando derecho. El operador de desplazamiento a la derecha desplaza los bits de su operando izquierdo hacia la derecha en el número de bits especificado por su operando derecho. El operador de complemento a nivel de bits define en el resultado todos los bits 0 en su operando a 1, y define todos los bits 1 a 0 en el resultado. En

Operador	Descripción
& AND a nivel de bits	Los bits en el resultado se establecen a 1 si los bits correspondientes en ambos operandos son ambos 1.
OR inclusivo a nivel de bits	Los bits en el resultado se establecen a 1 si por lo menos uno de los bits correspondientes en los dos operandos es 1.
^ OR exclusivo a nivel de bits	Los bits en el resultado se definen a 1 si exactamente uno de los bits correspondientes en los dos operandos es 1.
<< desplazamiento a la izquierda	Desplaza los bits del primer operando hacia la izquierda en el número de bits especificado por el segundo operando; rellena a partir de la derecha con bits 0.
>> desplazamiento a la derecha	Desplaza los bits del primer operando hacia la derecha en el número de bits especificado por el segundo operando; el método de rellenar a partir de la izquierda depende de la máquina.
~ complemento a uno	Todos los bits 0 se definen a 1 y todos los bits 1 se definen a cero.

Fig. 10.6 Los operadores a nivel de bits.

los ejemplos que siguen aparecen análisis detallados de cada operador a nivel de bits. Los operadores a nivel de bits se resumen en la figura 10.6.

Al utilizar los operadores a nivel de bits, es útil imprimir los valores en su representación binaria, para ilustrar los efectos precisos de estos operadores. El programa de la figura 10.7 imprime un entero **unsigned** en su representación binaria en grupos de ocho bits cada uno. La función **displayBits** utiliza el operador AND a nivel de bits para combinar la variable **value** con la variable **displayMask**. A menudo, el operador AND a nivel de bits se utiliza con un operando conocido como una *máscara* —un valor entero con bits específicos establecidos a 1. Las máscaras se utilizan para ocultar algunos bits en un valor, mientras otros bits se seleccionan. En la función **displayBits**, la variable de máscara **displayMask** es asignada el valor **1 << 15** (**100000000 00000000**). El operador de desplazamiento a la izquierda desplaza el valor

```
/* Printing an unsigned integer in bits */
#include <stdio.h>

main()
{
    unsigned x;
    void displayBits(unsigned);

    printf("Enter an unsigned integer: ");
    scanf("%u", &x);
    displayBits(x);
    return 0;
}

void displayBits(unsigned value)
{
    unsigned c, displayMask = 1 << 15;

    printf("%7u = ", value);

    for (c = 1; c <= 16; c++) {
        putchar(value & displayMask ? '1' : '0');
        value <<= 1;

        if (c % 8 == 0)
            putchar(' ');
    }

    putchar('\n');
}
```

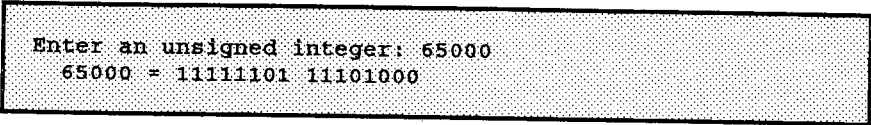


Fig. 10.7 Cómo imprimir un entero no signado en bits.

1 de la posición inferior (más a la derecha) hacia el bit de orden superior (más a la izquierda) en **displayMask**, y rellena con bits 0 a partir de la derecha. El enunciado

```
putchar(value & displayMask ? '1' : '0');
```

determina si deberá de imprimirse un 1 o un 0 para el bit actual más a la izquierda de la variable **value**. Suponga que la variable **value** contiene **65000** (**11111101 11101000**). Cuando se combinan **value** y **displayMask** utilizando **&**, todos los bits, a excepción del bit de orden superior, en la variable **value**, son “enmascarados” (ocultos) porque cualquier bit “manipulado por AND” con 0 da como resultado 0. Si el bit más a la izquierda es 1, **value & displayMask** se evalúa a 1, y 1 se imprime —de lo contrario se imprime 0. La variable **value** es después desplazada un bit a la izquierda, mediante la expresión **value << = 1** (este es equivalente a **value = value << = 1**). Estos pasos se repiten para cada uno de los bits en la variable **unsigned value**. En la figura 10.8 se resumen los resultados de combinar dos bits con el operador AND a nivel de bits.

Error común de programación 10.12

Usar el operador lógico AND (&&), en lugar del operador AND a nivel de bits (&), y viceversa.

El programa de la figura 10.9 demuestra el uso del operador AND a nivel de bits, del operador OR inclusivo a nivel de bits, del operador OR exclusivo a nivel de bits, y del operador de complemento a nivel de bits. El programa utiliza la función **display Bits** para imprimir los valores enteros **unsigned**. La salida se muestra en la figura 10.10.

En la figura 10.9, la variable entera **mask** es asignada al valor **1** (**00000000 00000001**), y a la variable **number1** se le asigna el valor **65535** (**11111111 11111111**). Cuando se combinan **mask** y **number1** utilizando el operador AND a nivel de bits (&) en la expresión **number1 & mask**, el resultado es **00000000 00000001**. Todos los bits, salvo el bit de orden inferior en la variable **number1** quedan “enmascarados” (ocultos), mediante la operación con el operador “AND” con la variable **mask**.

El operador OR inclusivo a nivel de bits se utiliza para definir en un operando bits específicos a 1. En la figura 10.9, la variable **number1** es asignada **15** (**00000000 00001111**), y la variable **setBits** es asignada **241** (**00000000 11110001**). Cuando se combinan **number1** y **setBits**, utilizando el operador OR a nivel de bits en la expresión **number1 | setBits**, el resultado es **255** (**00000000 11111111**). En la figura 10.11 se resumen los resultados de combinar dos bits con el operador OR inclusivo a nivel de bits.

Error común de programación 10.13

Usar el operador OR lógico (| |), en lugar del operador OR a nivel de bits (|), y viceversa.

Bit 1	Bit 2	Bit 1 & Bit 2
0	0	0
1	0	0
0	1	0
1	1	1

Fig. 10.8 Resultados de combinar dos bits mediante el operador AND a nivel de bits &.

```

/* Using the bitwise AND, bitwise inclusive OR, bitwise
   exclusive OR, and bitwise complement operators */

#include <stdio.h>

void displayBits(unsigned);

main()
{
    unsigned number1, number2, mask, setBits;

    number1 = 65535;
    mask = 1;
    printf("The result of combining the following\n");
    displayBits(number1);
    displayBits(mask);
    printf("using the bitwise AND operator & is\n");
    displayBits(number1 & mask);

    number1 = 15;
    setBits = 241;
    printf("\nThe result of combining the following\n");
    displayBits(number1);
    displayBits(setBits);
    printf("using the bitwise inclusive OR operator | is\n");
    displayBits(number1 | setBits);

    number1 = 139;
    number2 = 199;
    printf("\nThe result of combining the following\n");
    displayBits(number1);
    displayBits(number2);
    printf("using the bitwise exclusive OR operator ^ is\n");
    displayBits(number1 ^ number2);

    number1 = 21845;
    printf("\nThe one's complement of\n");
    displayBits(number1);
    printf("is\n");
    displayBits(~number1);

    return 0;
}

```

Fig. 10.9 Cómo utilizar el AND a nivel de bits, el OR inclusivo a nivel de bits, el OR exclusivo a nivel de bits, y el operador de complemento a nivel de bits (parte 1 de 2).

El operador OR exclusivo a nivel de bits (^) define cada bit en el resultado a 1, si *exactamente* uno de los bits, correspondiente en sus dos operandos, es 1. En la figura 10.9, las variables **number1** y **number2** se les asigna a los valores 139 (00000000 10001011) y 199 (00000000 11000111) respectivamente. Cuando se combinan estas variables con el operador OR exclusivo, en la expresión **number1 ^ number2**, el resultado es 00000000 01001100. En la figura 10.12 se resumen los resultados de combinar dos bits utilizando el operador OR exclusivo a nivel de bits.

```

void displayBits(unsigned value)
{
    unsigned c, displayMask = 1 << 15;

    printf("%7u = ", value);

    for (c = 1; c <= 16; c++) {
        putchar(value & displayMask ? '1' : '0');
        value <<= 1;

        if (c % 8 == 0)
            putchar(' ');
    }

    putchar('\n');
}

```

Fig. 10.9 Cómo utilizar el AND a nivel de bits, OR inclusivo a nivel de bits, el OR exclusivo a nivel de bits, y el operador de complemento a nivel de bits (parte 2 de 2).

```

The result of combining the following
65535 = 11111111 11111111
1 = 00000000 00000001
using the bitwise AND operator & is
1 = 00000000 00000001

The result of combining the following
15 = 00000000 00001111
241 = 00000000 11110001
using the bitwise inclusive OR operator | is
255 = 00000000 11111111

The result of combining the following
139 = 00000000 10001011
199 = 00000000 11000111
using the bitwise exclusive OR operator ^ is
76 = 00000000 01001100

The one's complement of
21845 = 01010101 01010101
is
43690 = 10101010 10101010

```

Fig. 10.10 Salida correspondiente al programa de la figura 10.9.

El operador de complemento a nivel de bits (~) define todos los bits 1 existentes en su operando a 0 en el resultado y define todos los bits 0 a 1 en el resultado —o de otra forma conocido como “tomar el *complemento a uno* del valor”. En la figura 10.9 la variable **number1** es asignada al valor 21845 (01010101 01010101). Cuando se evalúa la expresión **~number1** el resultado es (10101010 10101010).

Bit 1	Bit 2	Bit 1 Bit 2
0	0	0
1	0	1
0	1	1
1	1	1

Fig. 10.11 Resultados de combinar dos bits mediante el operador OR Inclusivo a nivel de bits |.

Bit 1	Bit 2	Bit 1 ^ Bit 2
0	0	0
1	0	1
0	1	1
1	1	0

Fig. 10.12 Resultados de combinar dos bits mediante el operador OR exclusivo a nivel de bits ^.

El programa de la figura 10.13 demuestra el operador de desplazamiento a la izquierda (<<) así como el operador de desplazamiento a la derecha (>>). La función `displayBits` se utiliza para imprimir los valores enteros `unsigned`.

El operador de desplazamiento a la izquierda (<<) desplaza los bits de su operando izquierdo hacia la izquierda, en el número de bits especificados en su operando derecho. Los bits desalojados a la derecha serán remplazados con 0s; los 1s que se desplazan hacia la izquierda se pierden. En el programa de la figura 10.13, la variable `number1` es asignada al valor 960 (00000011 11000000). El resultado de desplazar a la izquierda la variable `number1` 8 bits en la expresión `number1 << 8` es 49152 (11000000 00000000).

El operador de desplazamiento a la derecha (>>) desplaza los bits de su operando izquierdo hacia la derecha, en el número de bits especificado por su operando derecho. Ejecutar un desplazamiento a la derecha en un entero `unsigned` hace que los bits desalojados a la izquierda sean remplazados por 0s; los 1s desplazados hacia la derecha se pierden. En el programa de la figura 10.13, el resultado de desplazar hacia la derecha `number1` en la expresión `number1 >> 8` es 3 (00000000 00000011).

Error común de programación 10.14

El resultado de desplazar un valor queda indefinido si el operando derecho es negativo o si el operando derecho es más grande que el número de bits en el cual se almacena el operando izquierdo.

Sugerencia de portabilidad 10.7

El desplazamiento a la derecha es dependiente de la máquina. Desplazar a la derecha un entero signado, en algunas máquinas llena los bits desalojados con ceros y en otras con 1s.

Cada operador a nivel de bits (a excepción del operador de complemento a nivel de bits) tiene un operador de asignación correspondiente. Estos operadores de asignación a nivel de bits se muestran en la figura 10.14, y se utilizan de forma similar a los operadores de asignación aritméticos, presentados en el capítulo 3.

```
/* Using the bitwise shift operators */
#include <stdio.h>

void displayBits(unsigned);

main()
{
    unsigned number1 = 960;

    printf("\nThe result of left shifting\n");
    displayBits(number1);
    printf("8 bit positions using the ");
    printf("left shift operator << is\n");
    displayBits(number1 << 8);

    printf("\nThe result of right shifting\n");
    displayBits(number1);
    printf("8 bit positions using the ");
    printf("right shift operator >> is\n");
    displayBits(number1 >> 8);
    return 0;
}

void displayBits(unsigned value)
{
    unsigned c, displayMask = 1 << 15;

    printf("%7u = ", value);

    for (c = 1; c <= 16; c++) {
        putchar(value & displayMask ? '1' : '0');
        value <<= 1;

        if (c % 8 == 0)
            putchar(' ');
    }

    putchar('\n');
}
```

The result of left shifting
960 = 00000011 11000000
8 bit positions using the left shift opertor << is
49152 = 11000000 00000000

The result of right shifting
960 = 00000011 11000000
8 bit positions using the right shift operator >> is
3 = 00000000 00000011

Fig. 10.13 Cómo utilizar los operadores de desplazamiento a nivel de bits.

Operadores de asignación a nivel de bits	
&=	Operador de asignación AND a nivel de bits.
=	Operador de asignación OR inclusivo a nivel de bits.
^=	Operador de asignación OR exclusivo a nivel de bits.
<<=	Operador de asignación de desplazamiento a la izquierda.
>>=	Operador de asignación de desplazamiento a la derecha.

Fig. 10.14 Los operadores de asignación a nivel de bits.

En la figura 10.15 se muestra la precedencia y asociatividad de los varios operadores presentados hasta este punto en el texto. Se muestran de arriba hacia abajo, en orden decreciente de precedencia.

10.10 Campos de bits

C proporciona la capacidad de especificar o definir el número de bits en el cual se almacena un miembro `unsigned` o `int` de una estructura o de una unión —conocidos como un *campo de bits*. Los campos de bits le permiten una mejor utilización de la memoria, al almacenar datos en el mínimo número de bits requeridos. Los miembros de campos de bits *deben* ser declarados como `int` o `unsigned`.

Operador	Asociatividad	Tipo
() [] . ->	de izquierda a derecha	el mas alto
+ - ++ -- ! (tipo) & * ~ sizeof	de derecha a izquierda	unario
* / %	de izquierda a derecha	multiplicativo
+ -	de izquierda a derecha	aditivo
<< >>	de izquierda a derecha	de desplazamiento
< <= > >=	de izquierda a derecha	relacional
== !=	de izquierda a derecha	igualdad
&	de izquierda a derecha	AND a nivel de bits
^	de izquierda a derecha	negación
	de izquierda a derecha	OR a nivel de bits
&&	de izquierda a derecha	AND lógico
	de izquierda a derecha	OR lógico
?:	de derecha a izquierda	condicional
= += -= *= /= %= &= = ^= <<= >>=	de derecha a izquierda	asignación
	de izquierda a derecha	coma

Fig. 10.15 Precedencia y asociatividad de operadores.

Sugerencia de rendimiento 10.3

Los campos de bits ayudan a ahorrar almacenamiento.

Considere la siguiente definición de estructura:

```
struct bitCard {
    unsigned face : 4;
    unsigned suit : 2;
    unsigned color : 1;
};
```

La definición contiene tres campos de bits `unsigned` —`face`, `suit`, y `color`— utilizadas para representar una carta de un mazo de 52 cartas. Se declara un campo de bits, haciendo seguir a un nombre de miembro `unsigned` o `int` con un signo de dos puntos (:) y una constante entera que representa el *ancho* del campo, es decir, el número de bits en el cual queda almacenado el miembro. La constante que representa el ancho debe ser un entero entre 0 y el número total de bits utilizados para almacenar un `int` en su sistema. Nuestros ejemplos fueron probados en una computadora con enteros de dos bytes (16 bits).

La definición de estructura anterior indica que el miembro `face` está almacenada en 4 bits, el miembro `suit` en 2 bits y el miembro `color` en 1 bit. El número de bits se basa en el rango deseado de valores correspondiente a cada miembro de estructura. El miembro `face` almacena valores entre 0 (Ace) y 12 (Rey) —4 bits pueden almacenar un valor entre 0 y 15. El miembro `suit` almacena valores entre 0 y 3 (0 = Diamantes, 1 = Corazones, 2 = Tréboles y 3 = Espadas)— 2 bits pueden almacenar un valor entre 0 y 3. Finalmente, el miembro `color` almacena ya sea 0 (Rojo) o 1 (Negro) —1 bit puede almacenar ya sea 0 ó 1.

El programa de la figura 10.16 (cuya salida se muestra en la figura 10.17), crea el arreglo `deck`, que contiene 52 estructuras `struct bitCard`. La función `fillDeck` inserta las 52 cartas en el arreglo `deck`, y la función `deal` imprime las 52 cartas. Note que se tiene acceso a los miembros de campos de bits de las estructuras exactamente como cualquier otro miembro de estructura. El miembro `color` se incluye para tener la posibilidad de indicar el color de la carta en un sistema que permita despliegues en color.

Es posible especificar un *campo de bits sin nombre*, en cuyo caso el campo se utiliza en la estructura como un *relleno*. Por ejemplo, la definición de estructura

```
struct example {
    unsigned a : 13;
    unsigned : 3;
    unsigned b : 4;
};
```

usa como relleno un campo de 3 bits sin nombre —en estos tres bits no se puede almacenar nada. El miembro `b` (en nuestra computadora de palabras de 2 bytes) se almacena en otra unidad de almacenamiento.

Un *campo de bits sin nombre con ancho cero*, se utiliza para alinear el siguiente campo de bits en el límite de la nueva unidad de almacenamiento. Por ejemplo, la definición de estructura

```
struct example {
    unsigned a : 13;
    unsigned : 0;
    unsigned b : 4;
};
```

utiliza un campo sin nombre de 0 bits para saltarse los bits restantes (tantos como existan) de la unidad de almacenamiento en la cual está almacenado **a**, y alinear **b** con el límite de la siguiente unidad de almacenamiento.

Sugerencia de portabilidad 10.8

Las manipulaciones de campos de bits son dependientes de la máquina. Por ejemplo, algunas computadoras permiten que los campos de bits crucen límites de palabras, en tanto que otras no lo permiten.

Error común de programación 10.15

Intentar tener acceso a bits individuales de un campo de bits como si fueran elementos de un arreglo. Los campos de bits no son "arreglos de bits".

Error común de programación 10.16

Intentar tomar la dirección de un campo de bits (el operador **&** no puede ser utilizado en conjunción con campos de bits, porque éstos no tienen direcciones).

Sugerencia de rendimiento 10.4

Aunque los campos de bits ahorran espacio, su uso puede hacer que el compilador genere código en lenguaje de máquina de ejecución más lenta. Esto ocurre debido a que tener acceso a sólo porciones de una unidad de almacenamiento direccionable toma más operaciones en lenguaje de máquina. Esto es uno de los muchos ejemplos de los tipos de intercambios espacio-tiempo que ocurren en la ciencia de la computación.

10.11 Constantes de enumeración

C proporciona un tipo final, definido por el usuario, conocido como una *enumeración*. Una enumeración, introducida por la palabra reservada **enum**, es un conjunto de constantes enteras representadas por identificadores. Estas *constantes de enumeración* son, en efecto, constantes simbólicas, cuyos valores pueden ser definidos automáticamente. Los valores de un **enum** se inician con 0, a menos de que se defina de otra manera, y se incrementan en 1. Por ejemplo, la enumeración

```
enum months {JAN, FEB, MAR, APR, MAY, JUN, JUL, AUG, SEP,
             OCT, NOV, DEC};
```

crea un nuevo tipo en **enum months**, en el cual los identificadores son definidos automáticamente a los enteros 0 a 11. Para numerar los meses 1 a 12, utilice la enumeración siguiente:

```
enum months {JAN = 1, FEB, MAR, APR, MAY, JUN, JUL, AUG,
             SEP, OCT, NOV, DEC};
```

Dado que el primer valor de la enumeración anterior se define explícitamente en 1, los valores subsiguientes se incrementan en 1 dando como resultado los valores 1 hasta 12. Los identificadores en una enumeración deben ser únicos. En una enumeración el valor de cada constante de numeración puede ser establecido explícitamente en la definición, mediante la asignación de un valor al identificador. Varios miembros de una enumeración pueden tener el mismo valor entero. En el programa de la figura 10.18, la variable de enumeración **month** se utiliza en una estructura **for** para imprimir los meses del año del arreglo **monthName**. Note que hemos hecho **monthName[0]** la cadena vacía **""**. Algunos programadores pudieran preferir definir **monthName[0]** a un valor como *****ERROR***** para indicar que ocurrió un error lógico.

```
/* Example using a bit field */

#include <stdio.h>

struct bitCard {
    unsigned face : 4;
    unsigned suit : 2;
    unsigned color : 1;
};

typedef struct bitCard Card;

void fillDeck(Card *);
void deal(Card *);

main()
{
    Card deck[52];

    fillDeck(deck);
    deal(deck);

    return 0;
}

void fillDeck(Card *wDeck)
{
    int i;

    for (i = 0; i <= 51; i++) {
        wDeck[i].face = i % 13;
        wDeck[i].suit = i / 13;
        wDeck[i].color = i / 26;
    }
}

/* Function deal prints the cards in two column format */
/* Column 1 contains cards 0-25 subscripted with k1 */
/* Column 2 contains cards 26-51 subscripted with k2 */

void deal(Card *wDeck)
{
    int k1, k2;

    for (k1 = 0, k2 = k1 + 26; k1 <= 25; k1++, k2++) {
        printf("Card:%3d Suit:%2d Color:%2d  ",
            wDeck[k1].face, wDeck[k1].suit, wDeck[k1].color);
        printf("Card:%3d Suit:%2d Color:%2d\n",
            wDeck[k2].face, wDeck[k2].suit, wDeck[k2].color);
    }
}
```

Fig. 10.16 Cómo utilizar campos de bits para almacenar un mazo de cartas.

Card: 0	Suit: 0	Color: 0	Card: 0	Suit: 2	Color: 1
Card: 1	Suit: 0	Color: 0	Card: 1	Suit: 2	Color: 1
Card: 2	Suit: 0	Color: 0	Card: 2	Suit: 2	Color: 1
Card: 3	Suit: 0	Color: 0	Card: 3	Suit: 2	Color: 1
Card: 4	Suit: 0	Color: 0	Card: 4	Suit: 2	Color: 1
Card: 5	Suit: 0	Color: 0	Card: 5	Suit: 2	Color: 1
Card: 6	Suit: 0	Color: 0	Card: 6	Suit: 2	Color: 1
Card: 7	Suit: 0	Color: 0	Card: 7	Suit: 2	Color: 1
Card: 8	Suit: 0	Color: 0	Card: 8	Suit: 2	Color: 1
Card: 9	Suit: 0	Color: 0	Card: 9	Suit: 2	Color: 1
Card: 10	Suit: 0	Color: 0	Card: 10	Suit: 2	Color: 1
Card: 11	Suit: 0	Color: 0	Card: 11	Suit: 2	Color: 1
Card: 12	Suit: 0	Color: 0	Card: 12	Suit: 2	Color: 1
Card: 0	Suit: 1	Color: 0	Card: 0	Suit: 3	Color: 1
Card: 1	Suit: 1	Color: 0	Card: 1	Suit: 3	Color: 1
Card: 2	Suit: 1	Color: 0	Card: 2	Suit: 3	Color: 1
Card: 3	Suit: 1	Color: 0	Card: 3	Suit: 3	Color: 1
Card: 4	Suit: 1	Color: 0	Card: 4	Suit: 3	Color: 1
Card: 5	Suit: 1	Color: 0	Card: 5	Suit: 3	Color: 1
Card: 6	Suit: 1	Color: 0	Card: 6	Suit: 3	Color: 1
Card: 7	Suit: 1	Color: 0	Card: 7	Suit: 3	Color: 1
Card: 8	Suit: 1	Color: 0	Card: 8	Suit: 3	Color: 1
Card: 9	Suit: 1	Color: 0	Card: 9	Suit: 3	Color: 1
Card: 10	Suit: 1	Color: 0	Card: 10	Suit: 3	Color: 1
Card: 11	Suit: 1	Color: 0	Card: 11	Suit: 3	Color: 1
Card: 12	Suit: 1	Color: 0	Card: 12	Suit: 3	Color: 1

Fig. 10.17 Salida del programa de la figura 10.16.

Error común de programación 10.17

Es un error de sintaxis asignar un valor a una constante de numeración después de haber sido definida.

Práctica sana de programación 10.6

Utilice sólo letras mayúsculas en los nombres de las constantes de numeración. Esto hace que estas constantes destaquen en un programa y le recuerdan al programador que las constantes de numeración no son variables.

Resumen

- Las estructuras son colecciones de variables relacionadas, algunas veces conocidas como agregados, bajo un solo nombre.
- Las estructuras pueden contener variables de varios tipos de datos.
- La palabra reservada `struct` empieza toda definición de estructura. Dentro de las llaves de la definición de estructura, están las declaraciones de los miembros de la estructura.
- Los miembros de la misma estructura deben de tener nombres únicos.

```
/* Using an enumeration type */
#include <stdio.h>

enum months {JAN = 1, FEB, MAR, APR, MAY, JUN,
             JUL, AUG, SEP, OCT, NOV, DEC};

main()
{
    enum months month;
    char *monthName[] = {"", "January", "February", "March",
                        "April", "May", "June", "July",
                        "August", "September", "October",
                        "November", "December"};

    for (month = JAN; month <= DEC; month++)
        printf("%2d%11s\n", month, monthName[month]);

    return 0;
}
```

1	January
2	February
3	March
4	April
5	May
6	June
7	July
8	August
9	September
10	October
11	November
12	December

Fig. 10.18 Cómo utilizar una enumeración.

- Una definición de estructura crea un nuevo tipo de datos que puede ser utilizado para declarar variables.
- Existen dos métodos para declarar variables de estructura. El primer método es declarar las variables en una declaración, como se hace con las variables de otros tipos de datos, utilizando `struct tagName` como el tipo. El segundo método es incluir las variables encerradas en las llaves de la definición de estructura y en el punto y coma que termina la definición de estructura.
- El nombre de rótulo de la estructura es opcional. Si la estructura se define sin un nombre de rótulo, las variables del tipo de datos derivados deben de ser declarados en la definición de estructura, y no se pueden declarar otras variables del nuevo tipo de estructura.
- Una estructura puede ser inicializada con una lista de inicialización, siguiendo el nombre de la variable en la declaración de estructura con un signo igual y una lista de inicializadores, separados por comas y encerrados en llaves. Si en la lista existen menos inicializadores que

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE
DOCUMENTACION Y BIBLIOTECAS

miembros en la estructura, los miembros restantes serán automáticamente inicializados a cero (o a **NULL** si el miembro es un apuntador).

- Estructuras completas pueden ser asignadas a variables de estructura del mismo tipo.
- Una variable de estructura puede ser inicializada con una variable de estructura del mismo tipo.
- El operador de miembro de estructura se utiliza al tener acceso a un miembro de una estructura vía el nombre de la variable de estructura.
- El operador de apuntador de estructura —creado con un signo de menos (-) y un signo de mayor que (>)— se utiliza al tener acceso a un miembro de una estructura vía un apuntador a la estructura.
- Las estructuras y los miembros individuales de las estructuras se pasan a las funciones en llamada por valor.
- Para pasar a una estructura llamada por referencia, pase la dirección de la variable de estructura.
- Un arreglo de estructura se pasa automáticamente en llamada por referencia.
- Para pasar un arreglo en llamada por valor, cree una estructura con el arreglo como un miembro.
- Crear un nuevo nombre utilizando **typedef** no crea un nuevo tipo; crea un nombre que es un seudónimo del tipo anteriormente definido.
- Una unión es un tipo de datos derivado, cuyos miembros comparten el mismo espacio de almacenamiento. Los miembros pueden ser de cualquier tipo.
- El almacenamiento reservado para una unión debe ser lo suficientemente grande para almacenar su miembro mayor. En la mayoría de los casos, las uniones contienen dos o más tipos de datos. Solamente un miembro y, por lo tanto, un solo tipo de datos, pueden ser referenciados en un momento dado.
- Una unión se declara con la palabra reservada **union**, en el mismo formato que una estructura.
- Una unión puede ser inicializada únicamente con el valor del tipo de su primer miembro.
- El operador AND a nivel de bits (&) toma dos operandos integrales. Un bit en el resultado se define a 1 si los bits correspondientes en cada uno de los operandos son 1.
- Se utilizan máscaras para ocultar algunos bits mientras se conservan otros.
- El operador OR inclusivo a nivel de bits (|) toma dos operandos. Un bit en el resultado se define a 1 si el bit correspondiente en cualquier operando está definido a 1.
- Cada uno de los operadores a nivel de bits (a excepción del operador de complemento a nivel de bits unario) tiene un operador de asignación correspondiente.
- El operador OR exclusivo a nivel de bits (^) toma dos operandos. Un bit en el resultado se define a 1 si exactamente 1 de los bits correspondientes en los dos operandos está definido a 1.
- El operador de desplazamiento de izquierda (<<) desplaza los bits de su operando izquierdo hacia la izquierda en el número de bits especificados por su operando derecho. Los bits desalojados a la derecha se remplazan con 0s.
- El operador de desplazamiento a la derecha (>>) desplaza los bits de su operando izquierdo hacia la derecha en el número de bits especificado en su operando derecho. El ejecutar un desplazamiento a la derecha en un entero no signado hace que los bits desocupados a la izquierda sean remplazados por cero. Los bits desocupados en enteros signados podrían ser remplazados con 0s o con 1s —esto dependerá de la máquina.

- El operador de complemento a nivel de bits (~) toma un operando e invierte sus bits —esto produce el complemento a uno del operando.
- Los campos de bits reducen la utilización del almacenamiento al almacenar datos en el número mínimo de bits requeridos.
- Los miembros de campos de bits deben de ser declarados como **int** o **unsigned**.
- Un campo de bits se declara haciendo seguir un nombre de miembro **unsigned** o **int** con un punto y coma, y con el ancho del campo de bits.
- El ancho del campo de bits debe ser una constante entera, entre 0 y el número total de bits utilizados para almacenar una variable **int** en su sistema.
- Si un campo de bits se especifica sin nombre, el campo se utilizará como relleno en la estructura.
- Un campo de bits sin nombre con ancho 0 se utiliza para alinear el siguiente campo de bits en el límite de la siguiente palabra de máquina.
- Una enumeración, designada con la palabra reservada **enum**, es un conjunto de enteros que se representan mediante identificadores. Los valores de un **enum** se inician con 0, a menos de que se especifique lo contrario, y son siempre incrementados en 1.

Terminología

^ operador OR exclusivo a nivel de bits	nombre de miembro
^= operador de asignación OR exclusivo a nivel de bits	estructuras anidadas
~ operador de complemento a uno	complemento a uno
& operador AND a nivel de bits	relleno
&= operador de asignación AND a nivel de bits	apuntador a una estructura
operador OR inclusivo a nivel de bits.	tipos de datos definidos por el programador
= operador de asignación OR inclusivo a nivel de bits.	registro
<< operador de desplazamiento a la izquierda	desplazamiento a la derecha
<=< operador de asignación de desplazamiento a la izquierda	estructura autorreferenciada
>> operador de desplazamiento a la derecha	desplazar
>=> operador de asignación de desplazamiento a la derecha	intercambios espacio-tiempo
como tener acceso a miembros de estructuras	struct
agregados	asignación de estructura
arreglo de estructuras	declaración de estructura
campo de bits	definición de estructura
operador a nivel de bits	inicialización de estructura
complementar	operador de miembro de estructura
tipo derivado	(punto) (.)
enumeración	nombre de estructura
constante de enumeración	operador de apuntador de estructura
inicialización de estructuras	(flecha) (->)
desplazamiento a la izquierda	etiqueta de estructura
máscara	tipo de estructura
enmascarar bits	nombre de etiqueta
miembro	typedef
	union
	campo de bits sin nombre
	ancho de un campo de bits
	campo de bits de ancho cero

Errores comunes de programación

- 10.1 Olvidar el punto y coma que da por terminada una definición de estructura.
- 10.2 Asignar una estructura de un tipo a una estructura de un tipo distinto.
- 10.3 Es un error de sintaxis comparar estructuras, debido a diferentes requisitos de alineación en los diferentes sistemas.
- 10.4 Insertar un espacio entre el signo de - y el signo de > del operador de apuntador de estructura (o insertar espacios entre los componentes de cualquier otro operador múltiple de teclado, a excepción de ? :).
- 10.5 Intentar referirse a un miembro de una estructura utilizando únicamente el nombre de dicho miembro.
- 10.6 No utilizar paréntesis al referirse a un miembro de estructura utilizando un apuntador y el operador de miembro de estructura (por ejemplo `*aptr.suit`, es un error de sintaxis).
- 10.7 Suponer que las estructuras, como los arreglos, se pasan automáticamente en llamada por referencia, e intentar modificar los valores de estructura del llamador en la función llamada.
- 10.8 Olvidar incluir el subíndice de arreglo al referirse a estructuras individuales de un arreglo de estructuras.
- 10.9 Es un error lógico referenciar con el tipo equivocado, datos en una unión almacenados con un tipo distinto.
- 10.10 Es un error de sintaxis comparar uniones, debido a los diferentes requisitos de alineación en varios sistemas.
- 10.11 Inicializar una unión en una declaración con un valor cuyo tipo es diferente del tipo del primer miembro de la unión.
- 10.12 Usar el operador lógico AND (&&), en lugar del operador AND a nivel de bits (&), y viceversa.
- 10.13 Usar el operador OR lógico (| |), en lugar del operador OR a nivel de bits (|), y viceversa.
- 10.14 El resultado de desplazar un valor queda indefinido si el operando derecho es negativo o si el operando derecho es más grande que el número de bits en el cual se almacena el operando izquierdo.
- 10.15 Intentar tener acceso a bits individuales de un campo de bits como si fueran elementos de un arreglo. Los campos de bits no son "arreglos de bits".
- 10.16 Intentar tomar la dirección de un campo de bits (el operador & no puede ser utilizado en conjunción con campos de bits, porque estos no tienen direcciones).
- 10.17 Es un error de sintaxis asignar un valor a una constante de numeración después de haber sido definida.

Prácticas sanas de programación

- 10.1 Al crear un tipo de estructura proporcione un nombre de rótulo de estructura. El nombre de rótulo de estructura es conveniente más adelante en el programa para la declaración de nuevas variables de este tipo de estructura.
- 10.2 Seleccionar un nombre de rótulo de estructura significativo ayuda a autodocumentar el programa.
- 10.3 Evite utilizar los mismos nombres para miembros de estructura de distintos tipos. Ello es permitido, pero podría causar confusión.
- 10.4 No deje espacios alrededor de los operadores -> y . ya que ayuda a enfatizar que las expresiones en las cuales los operadores están contenidos son esencialmente nombres individuales de variables.
- 10.5 Ponga los nombres `typedef` en mayúsculas, para enfatizar que esos nombres son sinónimos de otros nombres de tipo.
- 10.6 Utilice sólo letras mayúsculas en los nombres de las constantes de numeración. Esto hace que estas constantes destaquen en un programa y le recuerdan al programador que las constantes de numeración no son variables.

Sugerencia de portabilidad

- 10.1 Dado que depende de la máquina el tamaño de los elementos de datos de un tipo particular, y debido a que las consideraciones de alineación de almacenamiento también son dependientes de la máquina, entonces también lo será la representación de una estructura.
- 10.2 Utilice `typedef` para ayudar a hacer más portátil un programa.
- 10.3 Si en una unión los datos se almacenan como de un tipo y se referencian como de otro tipo, los resultados serán dependientes de la instalación.
- 10.4 La cantidad de almacenamiento requerido para almacenar una unión es dependiente de la instalación.
- 10.5 En algunas uniones no pueden aplicarse fácilmente otros sistemas de computación. Si una unión es portable o no, a menudo depende de la alineación de almacenamiento requerida para los tipos de datos de miembros de unión en un sistema dado.
- 10.6 Las manipulaciones de datos a nivel de bits son dependientes de la máquina.
- 10.7 El desplazamiento a la derecha es dependiente de la máquina. Desplazar a la derecha un entero signado, en algunas máquinas llena los bits desalojados con ceros y en otras con 1s.
- 10.8 Las manipulaciones de campos de bits son dependientes de la máquina. Por ejemplo, algunas computadoras permiten que los campos de bits crucen límites de palabras, en tanto que otras no lo permiten.

Sugerencia de rendimiento

- 10.1 Es más eficaz pasar estructuras en llamada por referencia que pasar estructuras en llamada por valor (ya que esto último requiere que toda la estructura se copie).
- 10.2 Las uniones ahorran almacenamiento.
- 10.3 Los campos de bits ayudan a ahorrar almacenamiento.
- 10.4 Aunque los campos de bits ahorran espacio, su uso puede hacer que el compilador genere código en lenguaje de máquina de ejecución más lenta. Esto ocurre debido a que tener acceso a sólo porciones de una unidad de almacenamiento direccionable toma más operaciones en lenguaje de máquina. Esto es uno de los muchos ejemplos de los tipos de intercambios espacio-tiempo que ocurren en la ciencia de la computación.

Observación de ingeniería de software

- 10.1 Al igual que en una declaración `struct`, una declaración `union` simplemente crea un tipo nuevo. Colocar una declaración `union` o `struct` fuera de cualquier función no crea una variable global.

Ejercicios de autoevaluación

- 10.1 Llene los espacios en blanco con cada uno de los siguientes:
 - a) Una _____ es una colección de variables relacionadas bajo un nombre.
 - b) Una _____ es una colección de variables bajo un nombre en la cual las variables comparten el mismo almacenamiento.
 - c) Los bits en el resultado de una expresión utilizando el operador _____ se definen a 1 si los bits correspondientes en cada operando están establecidos en 1. De lo contrario, los bits se definen a cero.
 - d) Las variables declaradas en una definición de estructura se conocen como sus _____.
 - e) Los bits en el resultado de una expresión utilizando el operador _____ se definen a 1, si por lo menos uno de los bits correspondientes en cualquiera de los operandos está definido a 1. De lo contrario los bits se establecen a cero.
 - f) La palabra reservada _____ introduce una declaración de estructura.

- g) La palabra reservada `typedef` se utiliza para crear unseudónimo de un tipo de datos previamente definido.
- h) Los bits en el resultado de una expresión utilizando el operador `&` se definen a 1, si exactamente uno de los bits correspondientes en cada uno de los operandos está definido a uno. De lo contrario, los bits se definen a cero.
- i) El operador AND a nivel de bits `&`, se utiliza a menudo para `0` a los bits, esto es para seleccionar ciertos bits a partir de una cadena de bits, en tanto que se ponen a cero los demás.
- j) La palabra reservada `union` se utiliza para introducir una definición de unión.
- k) El nombre de la estructura se conoce como el `etiqueta` de la estructura.
- l) Se tiene acceso a un miembro de estructura ya sea con el operador `struct` o con el operador `ptr`.
- m) Los operadores `<<` y `>>` se utilizan para desplazar los bits de un valor hacia la izquierda o hacia la derecha, respectivamente.
- n) Una `enumeración` es un conjunto de enteros representados por identificadores.

10.2 Indique si cada uno de los siguientes es verdadero o falso. Si es falso, explique por qué.

- a) Las estructuras pueden contener únicamente un tipo de datos.
- b) Dos uniones pueden ser comparadas entre sí para determinar si son iguales.
- c) El nombre del rótulo de una estructura es opcional.
- d) Los miembros de diferentes estructuras deben tener nombres únicos.
- e) La palabra reservada `typedef` se utiliza para definir nuevos tipos de datos.
- f) Las estructuras se pasan siempre a las funciones en llamada por referencia.
- g) Las estructuras no pueden ser comparadas.

10.3 Escriba un solo enunciado o un conjunto de enunciados para llevar a cabo cada uno de los siguientes:

- a) Defina una estructura llamada `part` conteniendo la variable `int partNumber`, y el arreglo `char partName` cuyos valores pudieran ser de hasta 25 caracteres de largo.
- b) Defina `Part` como un sinónimo para el tipo `struct part`.
- c) Utilice `Part` para declarar la variable `a` que sea de tipo `struct part`, el arreglo `b[10]` que sea del tipo `struct part` y la variable `ptr` que sea del tipo apuntador a `struct part`.
- d) Lea un número de parte y un nombre de parte del teclado a los miembros individuales de la variable `a`.
- e) Asigne los valores del miembro de la variable `a` al elemento 3 del arreglo `b`.
- f) Asigne las direcciones del arreglo `b` a la variable de apuntador `ptr`.
- g) Imprima los valores de miembros del elemento 3 del arreglo `b`, utilizando la variable `ptr` y el operador de apuntador de estructura para referirse a los miembros.

10.4 Encuentre el error en cada uno de los siguientes:

- a) Suponga que `struct car` ha sido definido conteniendo dos apuntadores al tipo `char`, es decir, `face` y `suit`. También, la variable `c` ha sido declarada del tipo `struct card` y la variable `cPtr` ha sido declarada ser del tipo apuntador a `struct card`. La variable `cPtr` ha sido asignada a la dirección de `c`.

```
printf("%s\n", *cPtr->face);
```
- b) Suponga que `struct card` ha sido definida conteniendo dos apuntadores de tipo `char`, es decir `face` y `suit`. También, el arreglo `hearts[13]` ha sido declarado ser del tipo `struct card`. El siguiente enunciado debería imprimir el miembro `face` del elemento 10 del arreglo.

```
printf("%s\n", hearts.face);
```
- c) `union values {`

```
char w;
```

```
float x;
```

```
double y;
```

```
} v = {1.27};
```

```
d) struct person {
    char last Name[15]
    char firstName[15]
    int age;
}
```

- e) Suponga que `struct person` ha sido definido como en la parte (d) pero con la corrección apropiada.

```
person d;
```

- f) Suponga que la variable `p` ha sido declarada como del tipo `struct person` y la variable `c` ha sido declarada del tipo `struct card`.

```
p = c;
```

Respuestas a los ejercicios de autoevaluación

10.1 a) estructura. b) unión. c) AND a nivel de bits (`&`). d) miembros. e) OR inclusivo a nivel de bits (`|`). f) `struct`. g) `typedef`. h) OR exclusivo a nivel de bits (`^`). i) máscara. j) `union`. k) etiqueta. l) miembro de estructura, apuntador de estructura. m) operador de desplazamiento a la izquierda (`<<`), operador de desplazamiento a la derecha (`>>`). n) enumeración.

- 10.2 a) Falso. Una estructura puede contener muchos tipos de datos.
- b) Falso. Las uniones no pueden ser comparadas, debido a los mismos problemas de alineación asociados con las estructura.
- c) Verdadero.
- d) Falso. Los miembros de estructuras separadas pueden tener los mismos nombres, pero los miembros de una misma estructura deben tener nombres únicos.
- e) Falso. La palabra reservada `typedef` se utiliza para definir nuevos nombres (sinónimos) para tipos de datos definidos previamente.
- f) Falso. Las estructuras son siempre pasadas a las funciones en llamada por valor.
- g) Verdadero, debido a los problemas de alineación.

```
10.3 a) struct part {
    int partNumber;
    char partName[25];
};
b) typedef struct part Part;
c) Part a, b[10], *ptr;
d) scanf("%d%s", &a.partNumber, &a.partName);
e) b[3] = a;
f) ptr = b;
g) printf("%d %s\n", (ptr + 3)->partNumber,
    (ptr + 3)-partName);
```

- 10.4 a) Error: los paréntesis que deberían de encerrar a `*cPtr` han sido omitidos, causando que sea incorrecto el orden de evaluación de la expresión.
- b) Error: El subíndice del arreglo ha sido omitido. La expresión debería ser `hearts[10].face`.
- c) Error: Una unión solamente puede ser inicializada con un valor que tenga el mismo tipo que el primer miembro de la misma.
- d) Error: se requiere de un punto y coma para determinar una definición de estructura.
- e) Error: la palabra reservada `struct` fue omitida de la declaración de variable.
- f) Error: las variables de tipos de estructuras diferentes no pueden ser asignadas unos a los otros.

Ejercicios

10.5 Dé la definición de cada una de las siguientes estructuras y uniones:

- La estructura **inventory** que contiene el arreglo de caracteres **partName** [30], en entero **partNumber**, el punto flotante **price**, el entero **stock**, y el entero **reorder**.
- La unión **data** que contiene **char** **c**, **short** **s**, **long** **l**, **float** **f** y **double** **d**.
- Una estructura llamada **address** que contiene los arreglos de caracteres **street** **Address** [25], **city** [20], **state** [3], y **zipCode** [6].
- La estructura **student** que contiene los arreglos **firstName** [15] y **lastName** [15], y la variable **homeAddress** del tipo **struct address** correspondiente a la parte (c).
- La estructura **test** que contenga 16 campos de bits con anchos de 1 bit. Los nombres de los campos de bits son las letras **a** a la **p**.

10.6 Dadas las siguientes definiciones de estructuras y las declaraciones de variables,

```
struct customer {
    char lastName[15];
    char firstName[15];
    int customerNumber;

    struct {
        char phoneNumber[11];
        char address[50];
        char city[15];
        char state[3];
        char zipCode[6];
    } personal;
} customerRecord, *customerPtr;

customerPtr = &customerRecord;
```

escriba una expresión por separado que pueda ser utilizada para tener acceso a los miembros de la estructura en cada una de las partes siguientes.

- El miembro **lastName** de la estructura **customerRecord**.
- El miembro **lastName** de la estructura a la cual apunta **customerPtr**.
- El miembro **firstName** de la estructura **customerRecord**.
- El miembro **firstName** de la estructura a la cual apunta **customerPtr**.
- El miembro **customerNumber** de la estructura **customerRecord**.
- El miembro **customerNumber** de la estructura a la cual apunta **customerPtr**.
- El miembro **phoneNumber** del miembro **personal** de la estructura **customerRecord**.
- El miembro **phoneNumber** del miembro **personal** de la estructura a la cual apunta **customerPtr**.
- El miembro **address** del miembro **personal** de la estructura **customerRecord**.
- El miembro **address** del miembro **personal** de la estructura apuntada por **customerPtr**.
- El miembro **city** del miembro **personal** de la estructura **customerRecord**.
- El miembro **city** del miembro **personal** de la estructura a la cual apunta **customerPtr**.
- El miembro **state** del miembro **personal** de la estructura **customerRecord**.
- El miembro **state** del miembro **personal** de la estructura a la cual apunta **customerPtr**.
- El miembro **zipCode** del miembro **personal** de la estructura **customerRecord**.
- El miembro **zipCode** del miembro **personal** de la estructura a la cual apunta **customerPtr**.

10.7 Modifique el programa de la figura 10.16 para barajar las cartas utilizando un algoritmo de barajar de alto rendimiento (como se muestra en la figura 10.3). Imprima el mazo resultante en un formato de dos columnas, como en la figura 10.4. Anteceda cada carta con su color.

10.8 Crear la unión **integer** con miembros **char** **c**, **short** **s**, **int** **i**, y **long** **l**. Escriba un programa que introduzca el valor del tipo **char**, **short**, **int**, y **long**, y que almacene los valores en las variables de unión del tipo **union integer**. Cada variable de unión deberá ser impresa como un **char**, un **short**, un **int** y un **long**. ¿Se imprimen siempre los valores en forma correcta?

10.9 Crear la unión **floatingPoint** con los miembros **float** **f**, **double** **d**, y **long double** **l**. Escriba un programa que introduzca valor del tipo **float**, **double** y **long double**, y almacene los valores en variables de unión del tipo **union floatingPoint**. Cada variable de unión deberá imprimirse como un **float**, un **double** y un **long double**. ¿Se imprimen siempre los valores correctamente?

10.10 Escriba un programa que desplace una variable entera 4 bits hacia la derecha. El programa deberá imprimir el entero en bits antes y después de la operación de desplazamiento. ¿Su sistema coloca ceros, o bien unos en los bits desalojados?

10.11 Si su computadora utiliza enteros de 4 bytes, modifique el programa de la Figura 10.7, de tal forma que funcione con enteros de 4 bytes.

10.12 El desplazar a la izquierda un entero **unsigned** en 1 bit es equivalente a multiplicar el valor por 2. Escriba la función **power2** que toma dos argumentos enteros **number** y **pow** y calcule

$$\text{number} * 2^{\text{pow}}$$

Utilice el operador de desplazamiento para calcular el resultado. El programa deberá imprimir los valores como enteros y como bits.

10.13 El operador de desplazamiento a la izquierda puede ser utilizado para empaquetar dos valores de caracteres en una variable entera no signada de 2 bytes. Escriba un programa que introduzca dos caracteres del teclado y que los pase a la función **packCharacters**. Para empaquetar dos caracteres en una variable entera **unsigned**, asigne el primer carácter a la variable **unsigned**, desplace la variable a la izquierda en 8 posiciones de bits, y combine la variable **unsigned** con el segundo carácter utilizando el operador OR inclusivo a nivel de bits. El programa deberá extraer los caracteres en su formato de bits, antes y después de haber sido empaquetados en el entero **unsigned**, para probar que los caracteres de hecho han sido empaquetados correctamente en la variable **unsigned**.

10.14 Utilizando el operador de desplazamiento a la derecha, el operador AND a nivel de bits y una máscara, escriba la función **unpackCharacters** que toma el entero **unsigned** del Ejercicio 10.13 y lo desempaca en dos caracteres. Para desempacar dos caracteres de un entero **unsigned** de 2 bytes, combine el entero **unsigned** con la máscara 65280 (11111111 00000000) y desplace hacia la derecha el resultado en 8 bits. Asigne el valor resultante a una variable **char**. A continuación combine el entero **unsigned** con la máscara 255 (00000000 11111111). Asigne el resultado a otra variable **char**. El programa deberá imprimir el entero **unsigned** en bits, antes de ser desempacado, y a continuación imprimir los caracteres en bits para confirmar que fueron desempacados correctamente.

10.15 Si su sistema utiliza enteros de 4 bytes, vuelva a escribir el programa de Ejercicio 10.13 para empaquetar 4 caracteres.

10.16 Si su sistema utiliza enteros de 4 bytes, vuelva a escribir la función **unpackCharacters** del Ejercicio 10.14 para desempacar 4 caracteres. Crear las máscaras que necesite para desempacar los 4 caracteres desplazando hacia la izquierda el valor 255 en la variable de enmascaramiento en 8 bits 0, 1, 2 o 3 veces, (dependiendo del byte que está desempacando).

10.17 Escriba un programa que invierta el orden de los bits de un valor entero no signado. El programa deberá introducir el valor proveniente del usuario y llamar a la función `reverseBits` para imprimir los bits en orden inverso. Imprima el valor en bits tanto antes como después de la inversión de bits, para confirmar que los bits hayan sido invertidos correctamente.

10.18 Modifique la función `displayBits` de la Figura 10.7, de tal forma que resulte portátil entre sistemas, utilizando enteros de 2 bytes y sistemas de enteros de 4 bytes. *Sugerencia:* utilice el operador `sizeof` para determinar el tamaño de un entero en una máquina en particular.

10.19 El programa siguiente utiliza la función `multiple` para determinar si el entero introducido desde el teclado es un múltiplo de algún entero `X`. Examine la función `múltiple` y a continuación determine el valor de `X`.

```
/* This program determines if a value is a multiple of X */
#include <stdio.h>

int multiple(int);

main()
{
    int y;

    printf("Enter an integer between 1 and 32000: ");
    scanf("%d", &y);

    if (multiple(y))
        printf("%d is a multiple of X\n", y);
    else
        printf("%d is not a multiple of X\n", y);

    return 0;
}

int multiple(int num)
{
    int i, mask = 1, mult = 1;

    for (i = 1; i <= 10; i++, mask <<= 1)
        if ((num & mask) != 0) {
            mult = 0;
            break;
        }

    return mult;
}
```

10.20 ¿Qué es lo que ejecuta el siguiente programa?

```
#include <stdio.h>

int mystery(unsigned);

main()
{
    unsigned x;

    printf("Enter an integer: ");
    scanf("%u", &x);
    printf("The result is %d\n", mystery(x));
    return 0;
}

int mystery(unsigned bits)
{
    unsigned i, mask = 1 << 15, total = 0;

    for (i = 1; i <= 16; i++, bits <<= 1)
        if ((bits & mask) == mask)
            ++total;

    return total % 2 == 0 ? 1 : 0;;
}
```

Procesamiento de archivos

Objetivos

- Ser capaz de crear, leer, escribir y actualizar archivos.
- Familiarizarse con el proceso de archivos de acceso secuencial.
- Familiarizarse con el proceso de archivos de acceso directo.

Leo una parte en su totalidad.

Samuel Goldwyn

¡Saluden!

La bandera está pasando.

Henry Holcomb Bennett

La conciencia ... no se presenta a sí misma dividida en pedazos ...

Los símiles mediante los cuales se le puede describir más naturalmente serían un "río" o una "corriente".

William James

Debo suponer que un documento "No archivar" se archivará en un archivo "No archivar".

Senador Frank Church

Senate Intelligence Subcommittee Hearing, 1975

Sinopsis

- 11.1 Introducción
- 11.2 La jerarquía de los datos
- 11.3 Archivos y flujos
- 11.4 Cómo crear un archivo de acceso secuencial
- 11.5 Cómo leer datos de un archivo de acceso secuencial
- 11.6 Archivos de acceso directo
- 11.7 Cómo crear un archivo de acceso directo
- 11.8 Cómo escribir datos directamente a un archivo de acceso directo
- 11.9 Cómo leer datos directamente de un archivo de acceso directo
- 11.10 Estudio de caso: un programa de procesamiento de transacciones

Resumen • Terminología • Errores comunes de programación • Prácticas sanas de programación • Sugerencias de rendimiento • Sugerencias de portabilidad • Ejercicios de autoevaluación • Respuestas a los ejercicios de autoevaluación • Ejercicios.

11.1 Introducción

El almacenamiento de datos en variables y en arreglos es temporal; al terminar un programa todos estos datos se pierden. Para la conservación permanente de grandes cantidades de datos se utilizan los *archivos*. Las computadoras almacenan los archivos en dispositivos de almacenamiento secundario, especialmente en dispositivos de almacenamiento en disco. En este capítulo, explicaremos como los programas en C crean, actualizan y procesan los archivos de datos. Analizaremos tanto archivos de acceso secuencial como archivos de acceso directo.

11.2 La jerarquía de datos

En último término, todos los elementos de datos procesados por una computadora se reducen a combinaciones de ceros y de unos. Es así porque es simple y económico construir dispositivos electrónicos que puedan asumir dos estados estables —uno de los estados representando 0 y el otro 1. Es verdaderamente asombroso que las impresionantes funciones ejecutadas por las computadoras sólo involucren el manejo más básico de 0s y de 1s.

En una computadora el elemento de datos más pequeño puede asumir el valor 0 o el valor 1. Este elemento de datos se conoce como un *bit* (abreviatura de “binary digit” —un dígito que puede asumir uno de dos valores). Los circuitos de la computadora ejecutan varias manipulaciones simples de bits, como es determinar el valor de un bit, establecer el valor de un bit, e invertir un bit (de 1 a 0 o de 0 a 1).

Para los programadores resulta muy engorroso trabajar con datos bajo su forma de nivel bajo, es decir los bits. En vez de ello, los programadores prefieren trabajar con datos en forma de *dígitos decimales* (es decir 0, 1, 2, 3, 4, 5, 6, 7, 8 y 9), *letras* (es decir, de la A a la Z, y a hasta la z) y *símbolos especiales* (es decir, \$, @, %, &, *, (,), -, +, ", :, ?, /, y muchos otros). Los *dígitos*, *las letras* y *los símbolos especiales se conocen como caracteres*. El conjunto de todos los caracteres que pudieran

ser utilizados para escribir programas y representar elementos de datos de una computadora en particular se llama el *conjunto de caracteres* de dicha computadora. Dado que las computadoras sólo pueden procesar 1s y 0s, todo carácter de un conjunto de caracteres en una computadora es representado como un patrón de 1s y 0s (llamados un *byte*). Hoy día, los bytes están comúnmente formados por ocho bits. Los programadores crean programas y elementos de datos como caracteres; a continuación, las computadoras manipulan y procesan dichos caracteres como patrones de bits.

Al igual que los caracteres están formados por bits, los *campos* se componen de caracteres. Un campo es un grupo de caracteres que contiene un significado. Por ejemplo, un campo que consista únicamente de letras mayúsculas y minúsculas, puede ser utilizado para representar el nombre de una persona.

Los elementos de datos procesados por las computadoras forman una *jerarquía de datos*, en la cual los elementos de datos se convierten en más grandes y más complejos en cuanto a estructura conforme progresamos desde los bits, hacia los caracteres (bytes), hacia los campos y así sucesivamente.

Un *registro* (es decir, un *struct* en C) se compone de varios campos. En un sistema de nómina, por ejemplo, un registro para un empleado en particular pudiera estar formado por los campos siguientes:

1. Número de seguridad social
2. Nombre
3. Dirección
4. Tasa horaria de salario
5. Número de excepciones reclamadas
6. Ganancias acumuladas año a la fecha
7. Cantidades retenidas de impuestos federales, etcétera

Entonces, un registro es un grupo de campos relacionados. En el ejemplo anterior, cada uno de los campos corresponde al mismo empleado. Naturalmente, una compañía particular pudiera tener muchos empleados, y para cada uno de ellos tendrá un registro de nómina. Un *archivo* es un grupo de registros relacionados. El archivo de nóminas de una empresa normalmente contiene un registro para cada empleado. Entonces, un archivo de nóminas para una pequeña compañía pudiera contener únicamente 22 registros, en tanto que un archivo de nómina para una compañía grande pudiera contener 100,000 registros. No es desusado para una organización tener cientos e inclusive miles de archivos, muchos de ellos conteniendo millones y aún miles de millones de caracteres de información. Con la creciente popularidad de los discos laser ópticos y la tecnología de multimedia, pronto inclusive serán comunes archivos de billones de bytes. En la figura 11.1 se ilustra la jerarquía de los datos.

Para facilitar la recuperación de registros específicos a partir de un archivo, por lo menos un campo de cada registro es seleccionado como *registro clave*. Un registro clave identifica a un registro como perteneciente a una persona o entidad en particular. Por ejemplo, en el registro de nóminas descrito en esta sección, el número de seguridad social normalmente sería seleccionado como registro clave.

Existen muchas formas de organizar los registros dentro de un archivo. El tipo más popular de organización se conoce como *archivo secuencial*, en el cual típicamente los registros se almacenan en orden, en relación con el campo de registro clave. En un archivo de nóminas, los